

REPUBLIC OF TÜRKİYE
HASAN KALYONCU UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT



CO-OP FINAL REPORT

SUPERVISOR

Asst. Prof. Dr. Cihan KUZUDİŞLİ

BY

NURETTİN MUSTAFA ÖZKAYA

MAY 2026

ABSTRACT

My internship at Adım Yazılım, which I did through the BTK Academy program, allowed me to expand my knowledge and skills in the areas of database management, full-stack web development, and mobile application development.

The main part of the internship was about training on MS SQL Server. The topics covered were database architecture, SQL Server installation and configuration, T-SQL programming, indexing strategies, stored procedures, triggers, transaction management, backup and restore operations, and SQL Server Agent management. During this time, I also did the complete database schema for the TradeHub e-commerce platform, including tables for users, orders, products, payments, invoices, favorites, reviews, and order status tracking.

For the second stage, I made the web-based frontend of the TradeHub platform with Next.js App Router. I used server-side rendering, repository-based data management, and scalable UI components. I also built features like product listing, product detail pages, favorites, cart management, and an admin panel with role-based access control.

For the third stage, I developed a Flutter application for the mobile version of the platform. The app covers the full shopping experience, from product discovery and category filtering to checkout and order management.

To sum up, this internship made my backend development skills much stronger, gave me more analytical and problem-solving skills, and allowed me to experience the process of designing and delivering a complete, production-level e-commerce platform from database to web and mobile.

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS.....	3
LIST OF FIGURES	5
LIST OF SYMBOLS	6
CHAPTER 1: INTRODUCTION	7
1.1 Information on the Organization.....	7
1.2 Basic Activities on Adım Yazılım	7
1.3 Purpose of the Training and Project.....	7
CHAPTER 2: DETAILS OF ACTIVITIES.....	9
Practice and Application Activities During 1st Week	9
Practice and Application Activities During 2nd Week.....	10
Practice and Application Activities During 3rd Week	12
Practice and Application Activities During 4th Week.....	14
Practice and Application Activities During 5th Week.....	16
Practice and Application Activities During 6th Week.....	18
Practice and Application Activities During 7th Week.....	20
Practice and Application Activities During 8th Week.....	22
Practice and Application Activities During 9th Week.....	24
Practice and Application Activities During 10th Week.....	26
Practice and Application Activities During 11th Week.....	28
Practice and Application Activities During 12th Week.....	30
Practice and Application Activities During 13th Week.....	32
Practice and Application Activities During 14th Week.....	34
Practice and Application Activities During 15th Week.....	36
Practice and Application Activities During 16th Week.....	38
CHAPTER 3: INSTRUCTIONS RECEIVING AND GIVING PROCESS	39
3.1 Instructions Receiving Process.....	39
3.1.1 Instructions Receiving Task 1 – Database Design.....	39
3.1.2 Instructions Receiving Task 2 – Web Frontend Development.....	39
3.1.3 Instructions Receiving Task 3 – Mobile Application Development.....	40
CHAPTER 4: DISCUSSION AND CONCLUSION	41
REFERENCES	43

APPENDIX	44
<u>Appendix A – Week 1: SQL Server Setup & System Databases.....</u>	<u>44</u>
<u>Appendix B – Week 2: T-SQL Fundamentals & Table Design</u>	<u>46</u>
<u>Appendix C – Week 3: Advanced SQL & Indexing.....</u>	<u>47</u>
<u>Appendix D – Week 4: Transaction Management, Backup and Recovery.....</u>	<u>48</u>
<u>Appendix E – Week 5: Next.js UI/UX Edge Cases.....</u>	<u>49</u>
<u>Appendix F – Week 6: Favorites System & Cross-Page Consistency.....</u>	<u>50</u>
<u>Appendix G – Week 7: Checkout Flow</u>	<u>51</u>
<u>Appendix H – Week 8: Full Workflow & Integration.....</u>	<u>52</u>
<u>Appendix I – Week 9: Main, Account and Address Pages</u>	<u>53</u>
<u>Appendix J – Week 10: Product Detail, My Orders and Cart Flow</u>	<u>55</u>
<u>Appendix K – Week 11: Secure Checkout and Order Completion</u>	<u>56</u>
<u>Appendix L – Week 12: Admin Panel.....</u>	<u>57</u>
<u>Appendix M – Week 13: Mobile Product Discovery and Categories.....</u>	<u>59</u>
<u>Appendix N – Week 14: Mobile Product Detail and Favorites.....</u>	<u>60</u>
<u>Appendix O – Week 15: Mobile Secure Checkout and Order Flow</u>	<u>62</u>
<u>Appendix P – Week 16: Mobile Final Integration and End-to-End Testing.....</u>	<u>63</u>
<u>Appendix Q – Additional Screenshots</u>	<u>64</u>
<u>Appendix R – Proof of Work</u>	<u>66</u>
<u>Appendix S – Certificates</u>	<u>71</u>

LIST OF FIGURES

Figure 1 Mssql installation screenshot	44
Figure 2 Windows OS setup screenshot	44
Figure 3 SQL Server installation screenshot	45
Figure 4 SSMS launch screenshot	45
Figure 5 Sample database creation screenshot	46
Figure 6 Orders and OrderDetails Foreign Key Relationship.....	46
Figure 7 Normalization Overview (Diagram)	47
Figure 8 Clustered vs Non-Clustered Index test results.....	47
Figure 9 Clustered vs Non-Clustered Index	48
Figure 10 Initial state of the USERS table before transaction	48
Figure 11 Transaction rollback demonstration in the USERS table	49
Figure 12 Product Listing Page with Lazy-Loaded Images (UI Screenshot)	49
Figure 13 Lazy Loading Implementation (Relevant Code Snippet)	50
Figure 14 Favorites on product listing page	50
Figure 15 Product detail page.....	51
Figure 16 Cart UI.....	51
Figure 17 Checkout form with addresses	52
Figure 18 Search by Categories.....	52
Figure 19 Showing products for Search	53
Figure 20 Main Page	53
Figure 21 Account Page	54
Figure 22 Addresses Page.....	54
Figure 23 Order Page.....	55
Figure 24 Product Details Page	55
Figure 25 Cart Page	56
Figure 26 Secure Checkout Page.....	56
Figure 27 Order Received Page.....	57
Figure 28 Admin Dashboard	57
Figure 29 Admin Orders.....	58
Figure 30 Admin Order Details.....	58
Figure 31 Product list filtered by selected category.....	59
Figure 32 Screenshot of the category strip	59
Figure 33 Product detail page screenshot	60
Figure 34 Favorites screen.....	60
Figure 35 Secure Checkout screen	61
Figure 36 Cart screen.....	61
Figure 37 Order Successfully Screen	62
Figure 38 Account Page	62
Figure 39 Mobile Order Details Page.....	63
Figure 40 Orders Page.....	63
Figure 41 Profile screen.....	64
Figure 42 Reviews Screen	64
Figure 44 Review Page Web	65
Figure 43 Profile Page Web.....	65
Figure 45 Database Feedback.....	66
Figure 46 Develop both web and mobile platforms.....	66
Figure 47 Database Schema	67
Figure 48 Intern working on the TradeHub project during the internship period.....	68
Figure 49 General view of the Adım Yazılım office workspace during the internship	68
Figure 50 Intern actively developing the TradeHub project at the company workspace.....	69
Figure 51 Gaziantep Teknopark building where Adım Yazılım is located	70
Figure 52 Entrance of Adım Yazılım office at Gaziantep Teknopark.....	70
Figure 53 BTK Akademi Uygulamalarla SQL Öğreniyorum Certificate	71
Figure 54 Her Yönüyle SQL Server Certificate	71
Figure 55 BTK Akademi – React JS Next.js Certificate	72
Figure 56 Dart Programlama Dili Certificate	72
Figure 57 Flutter ile Mobil Uygulama Geliştirme Certificate	73

LIST OF SYMBOLS

Abbreviation	Description
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
API	Application Programming Interface
DB	Database
SQL	Structured Query Language
Next.js	React-based web framework
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol
HTTPS	Secure HyperText Transfer Protocol
JSON	JavaScript Object Notation
REST	Representational State Transfer

CHAPTER 1: INTRODUCTION

1.1 Information on the Organization

Adım Yazılım is a software development company that is located in Gaziantep Teknopark (Çamtepe Mah. Mahmut Tevfik Atay Bul. No: 4A/10, Şahinbey). They build ERP software systems, as well as desktop and mobile applications in addition to their range of services. The service that they provide includes the Paytahsilat service, which enables you to organize your finances using advanced software technology. There are currently ten employees at Adım Yazılım, including six computer engineers and four support staff, as well as three interns like me (myself). The IT professionals at Adım Yazılım provide skilled software engineers and developers with experience in mobile and web development. The hours of operation for Adım Yazılım are 08:30-18:00. Throughout my internship, I gained valuable experience working in an innovative environment applying state-of-the-art database management, web development, and mobile application development skills. The location of my internship can be seen in Figure 51, which depicts the Gaziantep Teknopark building.

1.2 Basic Activities on Adım Yazılım

Adım Yazılım is a software company that aims to develop scalable software products that meet the needs of organizations for digital transformation. The company serves businesses across various industries. The core of the company's business is software product development, which is the main focus of the organization's activities. One of the products like Paytahsilat can be known. Besides this, database management is another key area of the company's business. The specialists in this field are responsible for designing and optimizing databases built on MS SQL Server to handle large volumes of corporate data efficiently. In addition, the company offers services in e-commerce and web development, which include building advanced user-friendly web interfaces connected to back-end services.

1.3 Purpose of the Training and Project

My key aim when joining Adım Yazılım for my internship was the need to connect theory to practice and particularly learning more about relational database designing and full stack web and mobile applications development. There were three stages to be undertaken, one building up from the other, whereby knowledge gained from one phase would feed into the other.

The first phase was about achieving mastery of databases and specifically focusing on gaining a sound understanding of relational database principles and developing great T-SQL programming skills for professional database manipulation. In this phase, I achieved the following; database schema normalization, creation of stored procedures and triggers, transaction management, and backup and restore processes using Microsoft SQL Server.

Phase two focused on full stack web development, in which I developed a web platform called TradeHub, which was a full-fledged ecommerce platform using Next.js app router framework. This phase included the integration of database operations together with frontend technologies, server-side renderings, and creation of an end-to-end fully operational platform including the administration interface. This step involved taking the project into the mobile space by designing a Flutter application which would be an extension of the TradeHub shopping experience onto the mobile platform. In this way, this helped improve my knowledge of UI concepts for mobile applications, touch interactions, and ensuring data integrity between web and mobile application with access to the same database backend. Thus, through the completion of all these three steps, it ensured a coherent learning process in terms of covering the entire lifecycle of software product development.

CHAPTER 2: DETAILS OF ACTIVITIES

Practice and Application Activities During 1st Week

Week one was devoted to acquiring knowledge about database concepts as well as setting up my development environment. Week one was essential in establishing the technical background that is necessary for all other weeks of the internship to build upon. This week started with a broad introduction to database concepts including the relational model, data integrity, normalization, and database management system (DBMS) role in organizing and managing structured data. The explanation of Client/Server (C/S) architecture included a practical analogy so I could easily see how the database server receives, processes and returns responses to requests from clients. It was important for me to have this theoretical knowledge before moving to any practical work in the next phase of the internship. Once the theory portion of the week was completed, I began phase two (environment setup). The first step was to install VMware Workstation as my isolated virtual machine (VM) environment for SQL Server for the remainder of my internship. An operating system, Windows, was installed and configured on the VM. After installing and configuring Windows and using the VM, I installed and configured SQL Server and lastly, SQL Server Management Studio (SSMS), which is a graphical user interface (GUI) for communicating with the SQL Server (database engine). While configuring the server, I got used to using the SQL Server installation wizard, possible service accounts to choose from, and how to configure the database Engine. I also learned how to check that SQL Server was running correctly and how to connect for the first time locally via SSMS. I studied the four core system databases created automatically by SQL Server upon installation as part of the environment setup. Reviewing the master database is important because it contains all system-level configurations and login credentials necessary for the operation of the server; therefore, I considered this as the first core database to learn. Next, I looked at the model database because it is used as a template for creating every new user database, meaning all objects and settings defined in the model will automatically be inherited by newly created databases. Thirdly, I reviewed MSDB in that it contains all information associated with SQL Server Agent (job schedules, alerts, and backup history). Lastly, I studied tempdb because it is large, shared workspace for storing temporary tables, temporary results from queries and other objects that have a lifespan limited to the duration of the current user's session, are stored in this database.

Practice and Application Activities During 2nd Week

During the second week, participants focused on more T-SQL fundamental concepts; that would develop into the first tier of a traditional database development cycle. Participants moved from the configuration and the understanding of the concepts covered in Week 1, to the active development of databases and writing queries against those databases.

The week started out by studying and comparing all of the core SQL data types supported in SQL Server, specifically those for whole number storage types (INT and BIGINT), for both Standard and Unicode character storage (VARCHAR and NVARCHAR), and for Date and Time value storage (DATETIME and DATE). Participants looked at Decimal and Float Data Types for handling precise numeric values. Participants looked at and learned which of these Data types to use in each instance as incorrect choice may cause issues like inefficient data storage, loss of data or a broken query. After the data type study I moved into table creation with T-SQL CREATE TABLE statements. Primary Key constraints were applied to identify each record within a table uniquely, also I used Foreign Key constraints to put in place and enforce relationships between related tables. NOT NULL constraints were applied to columns that had to have a value, and also I used the UNIQUE constraint to preclude duplicate entries in fields like email and product codes. These constraints formed the base of our data integrity which I applied to all out following database designs One of the important things that were done during this week was learning how to write queries. During the week, I learned how to do simple queries using the SELECT statement from a single table before doing complex queries. I learned how to use the WHERE statement to filter results according to certain conditions and ORDER BY to order the results of the query in an ascending or descending fashion. GROUP BY combined with aggregate functions was discussed to find out ways to summarize results by grouping them into different categories. The HAVING clause was also introduced, which helps us filter results that have been grouped using aggregate functions.

JOINS were taught and worked on extensively throughout this week. The INNER JOIN was used to select only those records with matching data in the two tables being operated on, while LEFT JOIN was employed to retrieve all records from the left table as well as the matching records from the right table. The operation of RIGHT JOIN was also covered as the reverse of LEFT JOIN. The above JOINS were practiced on multi-tables featuring real-world related tables like employees, departments, orders, and customers.

I have independently created various sample databases in relation to relational databases

this week. This was done using some sample systems that include the company management system consisting of the following tables: employees, departments, and salaries; and the order management system comprising of the following tables: customers, orders, and products. This practice gave me ample opportunity to implement all the week's concepts in an actual database scenario, especially in the relationships of tables and JOIN operation of queries. Normalization has been covered as a proper method for minimizing data redundancy and structuring the databases. First, second, and third normal forms were considered and applied on the sample databases so that the data could be minimized and stored without any redundancy. The practice revealed that a bad decision made while designing tables could affect the database and cause update, insertion, and deletion anomalies.

A series of mistakes one might commit while operating with SQL Server has been detected by me during laboratory activities throughout the week. They comprise constraint violation due to a duplicated value of the primary key field; foreign key constraint violation since references can be made only to existing records; type mismatch mistake since wrong values are entered into certain fields. The analysis of the mistakes generated by SQL Server taught me how to perform debugging for this system and get acquainted with its operating principles. By the end of the second week, I managed to build a profound background in T-SQL programming, design of relational tables, formulation of queries, and data integrity enforcement techniques.

Practice and Application Activities During 3rd Week

The focus of the week three class was advanced SQL programming and ways to optimize the performance of databases. The concepts taught this week go above and beyond the basic writing of a query and designing a table in the two previous weeks. The concepts discussed this week include: Clustered and Non-Clustered Indexes, Views, User Defined Functions, Stored Procedures and Triggers All are basic building blocks for a professional database developer. The week opened with a deep dive into indexing strategies in SQL Server. The difference between Clustered and Non-Clustered Indexes was examined in detail. A Clustered Index determines the physical order in which the data is stored on disk and thus each table can have only one Clustered Index, which is usually set on the primary key column. A Non-Clustered Index, however, builds a separate structure that points back to the original data rows, which means multiple Non-Clustered Indexes can be created for a single table. Grasping this difference was crucial for deciding which columns to index depending on query patterns and how often they were accessed. The first thing that I did this week was a deep dive into Indexing Strategies with SQL Server. I looked at the difference between Clustered and Non-Clustered Indexes in detail. A Clustered Index is used to determine how the data will be stored on the disk i.e., each Table can only have one Clustered Index, typically associated with the Primary Key column. A Non-Clustered Index creates a separate structure that points to the original data rows, allowing multiple Non-Clustered Indexes to exist on one Table. Knowing the difference between each type of Index was key to making decisions about which Columns to Index based on how a query is written and how often data will be accessed. After completing the indexing study, I then studied Views, which are virtual tables that are created using a stored SELECT statement. Views were explored as a way of simplifying complex queries, restricting access to sensitive columns, and formatting data more meaningfully without duplicating the data. I created several Views based on the sample databases developed in the previous week to gain experience encapsulating multi-table JOIN queries and exposing only the necessary columns to the application layer. User Defined Functions were taken up next, with emphasis placed on two different kinds of User Defined Functions. First was the exploration of Scalar Functions that take input arguments and produce output as a single value, hence helping in wrapping common operations in order to reuse them in cases such as calculation of taxes and string manipulations. Another class of User Defined Functions was Table-Valued Functions, which generate a complete result set and can therefore be

thought of as views with the ability to apply dynamic filtering. In the following session, I will discuss user-defined functions: two major definitions. The first is a scalar function; a scalar function takes in an input parameter, performs its operation within the function body, and returns a single value. A scalar function may contain such reusable calculations as taxes or string formatting operations. The second type is a table-valued function; a table-valued function returns an entire result set; in this way, a table-valued function acts as a parameterized view and can be filtered dynamically at the function level. Both of these user-defined functions can be created and tested using sample data specific to the TradeHub platform. The week witnessed the concept of Stored Procedures being discussed thoroughly and their implementations put into practice. I designed Stored Procedures for each of the CRUD operations, namely inserting data, fetching data using optional filters, updating data, and deleting data while considering valid criteria to delete data. Input/Output parameters and error handling using TRY/CATCH mechanism were also discussed in relation to Stored Procedures. From this discussion, we can see that business logic implementation inside the database layer leads to increased reusability of code and simplification of application development, irrespective of which application communicates with the database. The final topic that the class learned was Triggers, which were presented as part of week three's lessons. The implementation of both AFTER and INSTEAD OF Triggers (for example: automatic data validation or logging) allowed me to see firsthand how Triggers can be used for business processes. An example would be that every time I made an INSERT, UPDATE or DELETE operation on key tables, I recorded these changes into my audit log tables using AFTER Triggers. This gives a documented history of data being modified. An INSTEAD OF Trigger is useful for intercepting and modifying the default operations performed by the database (like using Views with multiple base tables). Through the experience of implementing Triggers, I was able to see how automating processes at the database level positively affects data consistency, implements business rules, and provides an overall system-wide mechanism for controlling data-modifying events without changing my application code. By the end of Week three, my understanding of modular and maintainable database programming became far more sophisticated. The indexing concepts, logic implementations through Stored Procedures and Functions, automated implementations through Triggers, and easy data retrieval through Views formed a total package which could be employed to create the best possible database-level architecture. These acquired skills were then put to use in later weeks when designing the TradeHub database.

Practice and Application Activities During 4th Week

The fourth week of the database phase of the internship included a database-level enterprise concept, reliability and continuity of operations, and several of the previously mentioned concepts focused on a higher level of understanding of enterprise-level database systems. The topics for this week included Transaction Management, Handling Errors Comprehensively, Several Back-up Plans, Restore Operations, Automating through the SQL Server Agent, Several Database Maintenance Plans, Logical Cursor Management, and an Introduction to the Concepts of Big Data. During the week, an in-depth study of Transaction Management was done in SQL Server. Transactions were viewed as logical unit of works that involve many SQL transactions, with the result being such that either all of them succeed or none is applied on the database. Three main transaction control commands were learned. They include BEGIN TRANSACTION, which is used to define a point from which a transaction begins; COMMIT, which is used to permanently store all the transactions made; and ROLLBACK, which rolls back all the transactions made up to that point in time, thus reverting the database to its previous consistent state. In order to gain a more hands-on understanding of how transactions work, I carried out several experiments involving transactions by modifying data at different levels. I intentionally created errors in transactions (part-way through the transaction), did a rollback to see how ROLLBACK reverted data to its original state and I experimented with how SQL Server handles savepoints in nested transaction structures. This experience gave me a deeper understanding of the principles of ACID (atomicity, consistency, isolation and durability), which are necessary for reliable database systems:

Atomicity all operations in a single transaction will be treated as one cohesive operation.

Consistency - a database will only move from one valid state to another valid state.

Isolation concurrent transactions will not interfere with other concurrent transactions.

Durability once a transaction has been committed, all changes will be permanently retained even if the system fails. Researching error handling in transactions is one of the biggest areas of study; as such it makes sense that one of the best practices for production databases would be to use TRY / CATCH blocks for handling error occurrences during a transaction within the context of rollback operations when exceptions are thrown. The use of this pattern reduces the potential for a database to be corrupted by unexpected failures since partial (or incomplete) modifications to the data will not be committed to the database. Backup strategies were one more key aspect considered during the week. In total,

three types of back ups were discussed and practiced. First, Full Back Up was looked into as a comprehensive copy of the database on a certain moment in time and a basis for restoration operations. Next, Differential Back Up was viewed as a backup that saves only the data modified after the previous full backup; it is the best solution when there is a need to perform the same backup procedure frequently while saving time on it. Finally, Transaction Log Backup was introduced as the type of backing up a transaction log file that enables restoring the database to an exact point in time due to the information stored in the log file. All these types of backups were performed both in SSMS and with the help of T-SQL commands. During this week, SQL Server Agent (an automation tool for scheduling database tasks) was introduced and set up for use as the primary tool for automating some of the scheduled database tasks.

I have also successfully created and tested automated jobs that perform routine operations using the SQL Server Agent, such as creating scheduled backups, rebuilding indexes, and checking database integrity. Alert mechanisms were developed in order to notify administrators when certain conditions or errors thresholds are reached (e.g., if a scheduled job fails). Maintenance Plans were also examined as a higher-level scheduling option. They were used to schedule common maintenance tasks through a graphical user interface (similar to the SQL Server Agent jobs), providing greater detail about the scheduled task's progress and status. Cursors logic was explored as a technique of iterating on a set of records returned by queries on a row-by-row basis. Although cursors are often avoided when performance is a key factor because of their slower execution in comparison with set-based solutions, learning about the syntax and correct use of cursors in stored procedures was still important in order to handle certain exceptional cases. Finally, the course ended with an overview of Big Data ideas, which included an explanation of differences between traditional relational databases and new distributed architectures. Such issues as volume, velocity, and variety of data, as well as an introduction to relevant technologies, were covered in this discussion. Learning about Big Data gave valuable information about the position of SQL Server in the modern world. I learned, by the end of the fourth week, a lot of theoretical and practical application areas associated with storing data in an enterprise-level database and the associated tasks to be performed for maintenance, protection, and automation. The information provided me with a much better understanding of transaction management, backup plans, and scheduling automated tasks as it pertains to my decisions on structuring the TradeHub database schema to support production-level deployment environments.

Practice and Application Activities During 5th Week

The fifth week commenced the web frontend development phase of the internship, shifting the focus from database management to building the user interface of the TradeHub e-commerce platform using Next.js. This week focused specifically on boosting UI/UX robustness, treatment of edge cases, and enhancing frontend performance to ensure a stable and production-ready user experience.

The week began with an analysis of common edge cases that arise in e-commerce interfaces and how they should be handled seamlessly from a user experience perspective. One of the highly faced edge cases on product listing pages is when a particular search query returns no results by category filter. Rather than allowing a user to view nothing or a broken screen, I introduced separate empty state components that pass clear messages on-screen to make it obvious to the users that no results were found, improving the options to go back by adjusting the search term or browsing a different category. This approach significantly improved the overall usability of the platform by ensuring that any state of data always provided meaningful feedback to the user. Empty state design was used for other parts of the app such as favorites list, order history, and cart page. Each section would have displayed a contextual and informative message along with a call to action button in case there is no data. It helps the user to move ahead in the process without facing confusion with an inactive part of the application.

Another topic I worked on was performance optimization. Product listing pages in e-commerce applications often require displaying a number of product images at once. Such pages can be very slow to load initially if all the images are eagerly fetched. In order to improve performance, I decided to use the lazy-loading feature in product listing components using the Next.js's built-in image component. The image component allows for lazy loading, responsive image sizing, and optimized image formats out of the box, thus significantly lowering the page load time at the beginning when browsing categories with lots of products. In the past week, we learned about server-side rendering as part of the Next.js App Router architecture framework and how to put it into practice. One of the primary differences between server-side rendering (SSR) techniques compared with client-side rendering (CSR) methods is that with CSR, data fetching occurs after the browser receives the base HTML; whereas with SSR, all of the application's data is fetched and all of the necessary HTML is pre-rendered on the server before being sent to the client. Therefore, by using SSR, you can eliminate the flickering and layout shifting you see in

many CSR applications, where you're frequently waiting for just-in-time (JIT) component rendering, which occurs when a component renders without any data (e.g., blank or "loading") and then repaints when data has been returned from the API(s). Because the necessary data for rendering is fetched before rendering any components inside of server components, at TradeHub, when users visit the TradeHub frontend, they would be presented with 100% populated pages on page load; this allows TradeHub users to have a smooth and reliable experience when using the TradeHub frontend. Consistency within the User Interface between elements was also addressed and enhanced within this particular week. This included reviewing the consistency of the spacing, typography, color palette, and interactions across elements like the product listing card, recommended section, and category strip despite what information was being displayed within these elements. Specifically, this meant ensuring that regardless of how long or short the text used for each product title and description, it would not disrupt the layout of the card element. I explored a "Render-before-Response" pattern which will use Server-side Rendering by having the Server wait to send back a Fully Rendered page until all Data Fetch requests have completed. This I did in the lines of what has been done with Streaming and Suspense approaches we looked into. We went over each page in TradeHub which we looked at to present the best Rendering Strategy for each page. By Week 5 we had empirical proof from our hands on work of the benefits of putting Server/ Site Rendering together with what we did in terms of Edge Case Handling and Performance Tuning. What we put in this week gave us a very stable and very efficient Front End Platform to build on for the more complex features we worked out in the later weeks.

Practice and Application Activities During 6th Week

The sixth week of the project was spent on ensuring that the website is consistent cross-pages as well as making the favorite system more dependable in the entire TradeHub website. This was achieved by working on the architectural aspect that comes from thinking about the shared features and data states functioning consistently across the multiple Routes and Components from a holistic perspective, rather than only developing individual Pages in isolation. The week started with a review of the existing implementation of favorites, and specifically an analysis of the inconsistencies between favors for different Pages. Until this week' the favorites functionality had been developed in isolation on separate Pages so that a given item's favorite status could show differently when viewed from different Pages. For example, an item marked as favorite on the Product Details Page may not show as being favorite when viewed from the Product Listings Page or the Shopping Cart Page. To resolve this inconsistency, a single, centralized model was developed for managing the favorite State of products across all Pages displaying products. The solution used in this week consisted of server-side data fetching when the list of all item IDs saved in favorites for the currently authenticated user gets fetched once per request at the page level. Instead of sending the entire favorite object through the component hierarchy, the set of fetched IDs got transformed into the JavaScript Set, providing an ability to check its contents in constant time regardless of its size. Then, this Set gets passed as a prop in the FavoriteButton UI component, allowing us to check whether this product was added to favorites by looking for the presence of this product ID in the Set. This solution ensures consistency and efficiency since there is no need for database queries on the component level anymore. During the week, we refactored the FavoriteButton toward being context-independent and reusable across all scenarios including product listing cards, product detail pages, the dedicated favorites page, and the cart page without any modifications to its existing logic. All that the component requires as inputs are a productId and a favorites Set. All toggle interactions are submitted as server action requests to update the database and, subsequently, revalidate the necessary cached data. Therefore, any modification made to the favorites state on any page will reflect across all pages the next time the user accesses those pages, thus providing overall application consistency.

We also reviewed and improved the dedicated favorites page during the week. The page

will pull through the entire list of favorited products for the current user from the database and use the same product card components that we use on our listing pages to present the products visually the same across the application. We validated that we are properly providing the appropriate empty state message to users who have no saved favorites and are providing a proper navigation prompt. One of the major new features added to the platform this week was a personalized recommendations section on the cart page. This section of the cart page is dedicated to displaying a small selection of curated products, which will be displayed in a 'you may also like' panel, to give users even more products to consider while reviewing the contents of their cart. The recommendations displayed will be based on data that has been obtained from the server by way of datasets, such as best-selling products and most recently viewed products. One important requirement for this feature is to ensure that products already contained in the user's cart do not appear as part of the recommendation list, which would make the recommendations useless by virtue of being redundant – this filtering logic was accomplished server-side by comparing product IDs from the current cart with product IDs in the recommendation dataset fetched from the server, and removing all product records that were found in both datasets before returning results back to the rendering component. Component reusability was a common concept that ran throughout the week. All components built during the week incorporated a reusable design, which meant that these components were able to be included on any page without needing to duplicate any business logic. It was the responsibility of the page or layout to fetch data from the server, which meant that only the data needed for rendering would be passed to the presentational components, making it easier to scale as we build more pages later in the week. Server-side data composition emerged as an important aspect of the week since we had to fetch different datasets at once using the `Promise.all` method and filter the results. It became the recommended way to fetch datasets for all future pages that require multiple datasets from various sources. Component reusability was a common concept that ran throughout the week. All components built during the week incorporated a reusable design, which meant that these components were able to be included on any page without needing to duplicate any business logic. It was the responsibility of the page or layout to fetch data from the server, which meant that only the data needed for rendering would be passed to the presentational components, making it easier to scale as we build more pages later in the week. Server-side data composition emerged as an important aspect of the week since we had to fetch different datasets at once using the `Promise.all` method and filter the results.

Practice and Application Activities During 7th Week

During week seven, we will be focusing on completing the entire checkout process for TradeHub. We will cover a variety of topics, including user authentication/authorization-based access control, server-side price calculations, and handing off user information from the e-commerce platform into the checkout user interface. This week also represents one of the most important development phases for the eCommerce front-end of TradeHub. The checkout area is the last step of the e-commerce user experience and the most sensitive area of the shopping experience; therefore, it must work properly, dependably, safely, and uniformly under any circumstances. To start off the week, work on authentication-based routing for the checkout page was completed. For an online store, allowing users to visit the checkout page without being authenticated poses two potential issues. First, this raises concerns about security because accessing the checkout page requires the use of user data, including saved addresses, purchase history, and other personal information. Therefore, the checkout page had to be secured using the authentication status of the user on the server side. If the user had not authenticated himself or herself at the time of visiting the checkout page, they would be automatically redirected to the login page via Next.js routing while storing the original checkout URL in the route parameter to return back to it once the user completes the login procedure.

Price Calculations on the Server The price calculation required to complete the TradeHub will be done only on the Server in our Checkout Process. This way the user cannot manipulate the prices before sending them to the Server for processing. Example: Subtotal will be calculated based on the current cart items from the database. For each item, the quantity will be multiplied by the unit price in the database to derive an accurate subtotal. Shipping fees will also be based on predefined business rules on the server – and the Subtotal will be summed with the Shipping fees on the server before being sent back to the Checkout page to display to the user. Thus, no user has ever been able to manipulate the price of an order in the TradeHub. The integration of user address was completed within the week, to fulfil the delivery information section of the checkout form. The checkout page was designed to pull information from the database for all available saved delivery addresses for the authenticated user, and display those addresses in the form in an organized and selectable way. When an authenticated user accessed the checkout page, he/she could choose from one of his/her previously saved delivery addresses, or use the default delivery address they had previously selected. In order to do this, the checkout page needed to use relational data functionality to join the

User account record with the associated address records, and to find the most current and accurate address information for that user. Attention was also given to ensuring that if an authenticated user does not have any previously saved delivery Addresses, he/she will receive an appropriate message on the checkout page prompting him/her to add a Delivery Address before proceeding with the checkout transaction. The design of forms and user interface was another essential part of this week's efforts. The checkout form was created in such a way that all the necessary details were displayed in a logical manner. These included options to choose the delivery address, view the order details, and select the mode of payment. The three separate blocks were created to offer instant feedback to users regarding the completion of the process by highlighting any missing detail before submitting the form. Minimalism in the form design makes it easier for users to focus during the checkout process. This week, we have implemented a fix for the empty cart scenario. If a user was able to get to the checkout page with an empty cart, then they would encounter a broken experience as no items would be available for processing during checkout nor for calculating total prices. To fix this problem, a server-side check has been added to the checkout page to verify that the user has at least one item in their cart prior to rendering out the checkout form. In the event that a user attempts to view the checkout form with a completely empty cart, a friendly message will appear on the page informing the user that their cart is empty, along with a button that provides an easy method to return to the product catalogue and continue shopping. The implementation of this test will ensure that a user never has an encounter with an empty checkout form on the website. Error handling and edge case management were applied throughout the checkout flow to ensure robustness under unexpected conditions. Consideration for scenarios where the item is no longer available once it has been placed in the cart until the time when the check out process is being done have been made by providing fallback message for the same. Error-handling has been used on server-side functions which submit the check out form to avoid uncaught exceptions. The checkout process is quite resilient. It does not only consider the case when a product gets out of stock after the customer has added the item to their shopping cart and before the customer actually gets to the checkout page but has: At the same time, error handling and the management of any unforeseen situations (edge cases) that always occur when posting the order form through server actions, must be done consistently. This is so that the failures will be nicely communicated to the user instead of the unhandled exceptions being thrown to the user. The TradeHub web application had a complete, functioning, and protected checkout process by the seventh week.

Practice and Application Activities During 8th Week

In week eight, the main emphasis was on validation, end-to-end testing, and integration of all the workflows for the TradeHub e-commerce application. This was the last week of my internship on web frontend themes. I was ensuring that all modules, routes, and user flows built in the past weeks worked as one completely integrated application, as well as correctly, consistently, and reliably. My team and I started with a structured method for doing end-to-end workflow testing by testing the entire user journey from the moment they discover a product on our site until they complete their order. The flow of testing was based on following the most realistic user journey versus testing all individual components of the application in isolation. The first workflow we tested was the product discovery process—how users find products through the homepage, featured products, by category, or through search. All of those entry points into finding products had to be tested for valid product(s) displayed, correct filtering and sorting were applied, and pagination worked correctly for both the first or last items in a list of search results. The next test was conducted to examine the flow of adding products to favorites and performing verification for this operation on the favorites page, deleting products from the favorites, and verifying the removal in all pages with the product's presence. In light of the new design of favorites developed in the sixth week, the consistency was crucial in this test, and all cases confirmed that the unified favorites' Set model worked fine at all routes. The flow of cart management tests included such actions as adding products to the cart from listing and details pages, updating item quantity in the cart, deleting specific items, and checking the dynamic update of cart totals and item amounts after these changes. Another feature of cart management – the recommendations provided below the items added to cart – was also checked. The checkout flow has gone through the most rigorous testing of the week because it is the last part of the purchasing process and therefore very important to getting a transaction completed. The total checkout path was followed through multiple times, with multiple combinations for saved addresses and payment methods (Credit card, cash, etc.) to verify that every combination works correctly. Verification of the server-side price calculation was performed by comparing the displayed subtotal, shipping costs and total to manually calculated expected values generated from the database. The testing for authentication enforcement was done by accessing the checkout page while signed in as well as signed out, confirming that users who were not signed in were redirected to the login page and returned to complete the checkout flow after logging in. Edge Case &

Boundary Testing had been an independent component of this week's work. For testing the empty cart scenario, it was verified that if there were no items in your cart when you attempt to checkout, the correct messaging is shown within the checkout page as well as the proper redirect, rather than displaying a broken/blank form. The state of an empty favorites list was verified for both the Favorites page and for the FavoriteButton components on listing pages. Pagination boundary conditions were tested by navigating to both the first and last page of the product listings and ensuring the behavior of the previous and next navigation controls were functioning correctly at both limits by disabling appropriately when either the previous or next page do not exist. Also, to ensure that the empty state components created in week 5 rendered correctly based on all combinations of filters and/or keywords, we tested search queries that don't return any results. Server-side data consistency verification was performed for all order-related pages, including the list page of orders as well as their details page. Each page was checked to ensure that orders' statuses, number of products in an order, prices of items per piece, totals for lines, delivery addresses, as well as the total cost of orders were generated based on database data consistently. Specifically, orders consisting of more than one item were verified for accurate individual and overall calculations of information. In addition, any change made to the orders was displayed on both list and details pages without page refreshes. The admin panel was also checked for proper functioning during this week. Pages allowing users to manage orders, administer products list, and oversee accounts of other users were each tested to make sure that role-based access restrictions were working appropriately and all actions taken by admins were committed to the database accurately and immediately reflected on user pages. The last part of week eight was focused on completing the internship report and writing about the overall architecture of the system as well as the workflow of the application. In addition to describing the overall architecture and workflow of the system at a high level, a detailed description of how the Next (Next.js) App Router, server-side rendering, a repository pattern (data access), and an SQL Server backend work as an integrated system was included. In addition to providing a high-level overview of each pattern used in the architecture, a detailed description had to be created for the repository pattern, including the use of repository classes to wrap all database operations for a single domain entity and to provide a clear interface to the page and component layer.

Practice and Application Activities During 9th Week

During the ninth week we focused on overhauling and greatly improving the user account area of the TradeHub platform. We aimed to make the account management experience not only more organized and easy to maintain but also more enjoyable for users in terms of usability between the web and mobile interfaces. Also this week marked a major milestone for an intern which saw his work shift away from the primary shopping functions to the support of the account backend which in turn enabled a personal shopping journey on the e-commerce platform. At the start of the week we looked at present account page layout and identified the key issues which were of great dissatisfaction to the users. To date the account section has been a one stop shop for a user which included all info and action related to his/her account. This is to say that as the account features grew the user interface became a very messy and confusing picture. The account zone was redesigned using a modular approach that breaks the parts of managing an account down into four separate pages: Profile, Orders, Addresses, and Saved Cards. This modular design provides improved usability/clarity for users by providing direct access to the page they want, instead of having to search for their desired page among several different types of data, within one location. As well as improving the usability and clarity of the account zone, there were revisions made to the navigation that links the different account zones together. An account navigation menu, which has a consistent design throughout the account zone, has been created to allow quick access to each of the zones without requiring the user to go back to the top level of the account zone between actions from one account zone to another (Profile, Orders, Addresses, and Saved Cards). The routing used for this navigation was based on existing e-commerce practices used by major online retailers, which has resulted in a more professional and efficient user experience. Handling profile data was a significant issue in the campaign. The profile page was made to show and allow changes in all main user data fields, such as full name, email address, gender, date of birth, and phone number. A major point of the profile page was that all fields are to be filled with the user's current data first, when the page is opened, rather than showing a blank form that the user would have to fill in from the beginning. To make this work, the stored session data, which has the authenticated user's main identity information, was combined with local state management in the profile form component. The pre-filling logic was done so that each form field is set up with the relevant value loaded from the session or database on page load, but the user is still able to change any field before submitting the updated

information. The session data and shown profile information were kept in sync so that changes made through the profile page were instantly visible in all parts of the application showing user information, e.g., account navigation header and checkout delivery section. The address management component has been completely restructured during the last week to facilitate the entire database to be used in conjunction with the Address Management Web Application. Previously, the address data was being stored salt in two different methods which were local storage and session state. This created inconsistencies with what was displayed in each session as well as from the web versus the mobile environment. The redesigned address page now calls the database for all delivery addresses for the authenticated user each time the page is accessed so the current address exists for the user and accurately represents each new, updated, or deleted address made through either web or mobile. The add address and edit address forms have been created using server-side actions to persist the updates to the database and use the page revalidation to update the displayed address list after each action. The ability for users to select a preferred delivery address has also been implemented, enabling users to have a pre-selected delivery address on the checkout page. The handling of saved cards was linked to the database in the same way as address management. The saved cards page was built to fetch all payment card records linked with the logged-in user from the database and present them in an easily readable list. For each card, there was a card type, last four digits of the card number, and the expiry date displayed so that the user could distinguish between various payment cards added previously but without displaying any personal information. The functionality for adding cards as well as deleting them from the wallet was handled via server-side functions that updated the database and refreshed the page content accordingly. The Order History screen was modified to provide users with a detailed record of all orders placed previously by them. It shows order details such as order date, status, total cost, and list of items ordered. On clicking any particular order in the list, users were redirected to the Order Details screen, which would give them information about all aspects of their order, including items purchased with quantities and prices, delivery address, payment mode used, and order status timeline. Furthermore, the Orders tab in the account dashboard was also evaluated and enhanced. The Order History screen was modified to provide users with a detailed record of all orders placed previously by them. It shows order details such as order date, status, total cost, and list of items ordered.

Practice and Application Activities During 10th Week

The tenth week focused on expanding and refining the core product interaction features and shopping workflow of the TradeHub e-commerce application. This week covered a broad range of interconnected features including category-based filtering, product detail page development, favorites persistence, and comprehensive cart management enhancements, all of which contributed to delivering a more complete and realistic e-commerce experience for the end user. Started off the week with a re-design of category navigation area. In the older implementation, categories were displayed as a static list in which a user would have to click to separate pages when browsing products by their respective category. This was transformed into a horizontal strip view that used to showcase all of the product categories compactly and visually appealing right there on the main product listing screen. When the user tapped or clicked on a specific category, all products that fell under that category would be displayed in the product grid immediately below it, without reloading or leaving the page. The ability to discover products across multiple categories like this greatly improved the user experience of browsing for products, meeting or exceeding the expectations users would have about browsing for products on larger e-commerce sites. Mobile users' experience was enhanced further with the use of horizontal scrolling due to limited vertical screen space and as a result, many categories could be displayed using horizontal scrolling without using the limited screen space in an unwarranted manner. Once a user selects a product from any of the product listings (main product grid, category filtered view, search result, recommendations), the detail for that product would be displayed on a dedicated screen. This page was made to include all the relevant product information in an organized way and showcased it through a visual means. At the top of the page was included an image of the product along with its title and price, whereas right underneath it was the control that allowed the user to specify the quantity of the item being added to their cart. Request a Demo button was added towards the center of the page to ensure that the user would be able to see it regardless of how much they would scroll down the page in terms of accessing the product information. Along with the basic product details, there was a description section that offered more information about the product whereas two different cross-sell recommendation sections were included. The first cross-sell recommendation section included products in the same category as the product shown on the page, and the second one included products that

people buy with the current item. The creation of this page incorporated several reusable UI design concepts as all the controls that were present on the page, such as the product card and favorite/add to cart button, were reusable controls. This week we have significantly improved the Cart management in order to profit from a more complete pre-purchase workflow. Cart Page: Quantity Management Controls were added to the cart page which allowed users to increase or decrease quantity from within the cart itself, eliminating the need to go back and forth from product detail page. Then, a way to remove items individually from the cart were designed and developed, or clear the entire cart in one go. The cart page's order summary section was modified to recalculate the subtotal, estimated shipping price, and total amount dynamically based on any user-made changes in quantity or items. Every change to the cart was saved in the database using server actions — just like favourites system — so we had a consistent state that held between sessions and devices. This week we have significantly improved the Cart management in order to profit from a more complete pre-purchase workflow. Cart Page: Quantity Management Controls were added to the cart page which allowed users to increase or decrease quantity from within the cart itself, eliminating the need to go back and forth from product detail page. Then, a way to remove items individually from the cart were designed and developed, or clear the entire cart in one go. The cart page's order summary section was modified to recalculate the subtotal, estimated shipping price, and total amount dynamically based on any user-made changes in quantity or items. Every change to the cart was saved in the database using server actions — just like favourites system — so we had a consistent state that held between sessions and devices. In the tenth week, one of the likely most desired feature increments was the addition of much better product discovery and preparation experience feeds for shopping along with TradeHub application. The interactive filtering of categories, detailed product detail pages, favorites management that persisted and a fully functional cart workflow created an end-to-end shopping experience for the user from initial product discovery to purchase readiness.

Practice and Application Activities During 11th Week

In the eleventh week, the focus was on the development of the secure checkout process and the overall ordering flow creation for the mobile application TradeHub. As mentioned previously, this particular week was definitely one of the most difficult and crucial ones within the mobile development stage from the point of view of the technology used because the checkout process is the final stage of the purchasing process. This week started with developing the dedicated checkout page that would include all the necessary elements and stages of this procedure, as well as display all these elements in an orderly manner to facilitate the ordering process. It means that the page will have separate parts and will not contain all its information at once so that it does not overwhelm users with too much. First of all, this page included the stage of choosing the delivery address when a user had to select one of his/her previous addresses stored in the database or input the address manually while placing the order. The chosen delivery address was then presented in the form of a summary card. The payment methods section was created in order to accommodate various payment methods that can often be met when using the software in real-life situations. The user could pick a stored payment card from the drop-down menu; each of the stored cards would then be listed by their type, masked number, and expiration date. If a user wanted to use a payment card which had not been stored before, the new card entry area was available; it was filled in through appropriate input boxes for card number, cardholder name, expiration date, and security code. Other than card payments, two additional payment methods were offered as optional choices – bank transfer and cash on delivery. The way each of these payments was going to be conducted was briefly described right below the payment method selection. Although the actual payment processing in the current project was carried out only for demonstration purposes and did not actually perform any payments, the UI structure closely resembled that of the production environment. It was the order summary section on the checkout page that helped the user review the entire and comprehensive information about his/her order before submitting it. It contained the details of the cart items, including the images, item names, selected quantity, and individual prices per item. The section also featured the subtotal, shipping costs, and total costs. The order summary values were calculated on the server-side using the data directly fetched from the database. This made sure that the amounts were accurate and no one could change them by manipulating the client-side. The order summary was always available during the checkout process, so users could check

their orders at any time before placing the confirmation. Robustness has been implemented in the checkout process by thoroughly handling edge cases. Such cases as trying to go on without choosing a delivery address, submitting the checkout form while the cart is empty, server error occurrence during order creation have been tested and appropriate user feedback messages were provided. Validation was added to stop the order from being submitted if any required fields were missing, and the errors were displayed inline, so the user could easily identify and fill in the missing information. The order success screen has been created as the last stage of the checkout process, shown right after the completion of the order placement process. The purpose of the screen was to give a complete order confirmation and satisfaction to the user and to make the whole process seem completed professionally. The order number was shown as the first piece of information for reference purposes and then came the summary of the order that included items bought, their quantity and prices, as well as the total price of the order. Delivery address and payment method used were presented in order to confirm all details of the transaction that have been made. A part of the screen called 'Payment Explanation' explained how the payment should be done depending on which payment method the user chose. At the end of the page, there were two buttons – one to take the user to his orders in the orders tab of the account and another one to start shopping again. By the eleventh week, it had been possible to integrate the complete checkout procedure and order processing for the TradeHub application. This involved database support for addresses and payments, server-based ordering process, edge-case handling, and a dedicated screen to show the order success information. In this way, the checkout feature was made to be among the most comprehensive and production-like elements of the entire system. I found this experience to be quite enriching since it gave me an idea of how checkout mechanisms work and what considerations must be made in terms of post-order user experience.

Practice and Application Activities During 12th Week

In week twelve, and thus the final week of the course, we focused on being able to utilize the Admin Panel which was the final, required function of the TradeHub e-commerce system. This week was a very crucial transition since we paused our efforts on user-oriented functions and moved to developing the administration functions that enable a business to oversee and manage its platform. The specific tasks we accomplished for this week included designing the layout of the Admin panel to allow administrators to monitor Orders/Products; designing the workflow for managing the data in the databases; creating route security and role-based authorisation; Integrating the design of the Admin panel into the application. The dashboard of the Admin panel was created to give access to the Admin section as well as an overview of how the overall platform is operating. The dashboard includes various metrics to immediately give the administrator of the business a snapshot of the health of their business (i.e. total number of orders, total number of registered users, total number of products currently active, and recent activity of orders). By displaying key performance indicators in this manner, the Administrator will not need to visit several different pages in order to obtain simple operational information about their business. The next step of the administrative panel was order management section, being the most detailed and functionally filling component of admin interface. This part presented a full list of all orders on the platform which included order ID, customer name, order date, total price, order status and a link to full article view. We designed filter and sort features to better suit business needs which in turn enabled admin to narrow down order lists by status, date range or customer thus to handle a great volume of orders more effectively. In the admin panel we also put up order detail pages which went into full breakdown per order of line items including product names, quantities and prices, delivery address, payment method used and full order status history from the ORDER_STATUS_HISTORY table. Also we included the feature which gave admin to change an order status right on the detail page which in turn we recorded to the ORDER_STATUS_HISTORY table which in the end gave us a full and audit able trail of every business event related to order progress. For product management we created this section which the admins could use to see the full product catalog. Within the admin panel we put out a list of all products which we have in the database along with their categories, prices, stock levels and active status. This section was designed to support future extensions like adding new products, editing existing product details, and deactivating

products that no longer are available which at the same time was to lay the structure for a very functional product management workflow. Protection and restricted access of the routes were two crucial aspects that were considered during the creation of the admin panel. As all the pages of the admin panel include some confidential information about the company's activities, it was important to protect the routes in such a way that nobody could access them but those who have the administrator status. For this purpose, every admin page has the authentication and verification of the user's role performed right on the server side before the content rendering and data fetching starts. In other words, every time when the user accessed the admin routes without the administrator status, he or she got redirected to the main interface. The admin panel was integrated with the existing database-backed system in a way that respected the patterns established in the rest of the application. Any data shown in the admin panel was obtained through the same repository layer as the customer-facing pages and thus, the admin views were always up-to-date with the database. Server actions were responsible for all administrative write operations such as order status updates and this secure server-side data modification pattern was consistently used throughout the platform. By the end of the twelfth week, there was an admin panel which was not only fully operational but was also well secured and incorporated into the TradeHub system. The process of building the admin interface offered some practical understanding of how an ecommerce system in the production stage needs to cater to two very different users at the same time, one being the customer browsing and purchasing items, and the other the administrator monitoring, managing, and maintaining the system.

Practice and Application Activities During 13th Week

The internship's thirteenth week marked the start of the mobile app development phase. I shifted our focus from web front end to the development of the TradeHub e-commerce experience as a native mobile application with Flutter. The work done in this week mainly included the implementation of the mobile product discovery experience and the category-based browsing system, which laid the basic navigation and filtering patterns for the entire mobile shopping interface. The category navigation system was the first major feature developed during this week. In contrast to the traditional approach where categories were simply listed vertically or presented on a different page for navigation, the new solution included an ability to scroll through categories horizontally at the very top of the screen, where users would look for products to buy. The scrollable category strip featured all product categories in the form of chips that could be clicked on by users, who could now explore the entire collection of categories with a single horizontal scrolling motion. Dynamic width calculations of the chips meant that no matter how many categories there were – five or fifty – the strip layout would look neat and tidy. The implementation of category navigation has been the first significant feature developed within the span of this week. Instead of having a fixed vertical list of categories or having a separate navigation section, the developers chose to implement an horizontally scrollable category bar above the main content area where products could be discovered. All the product categories were represented in this bar through individual chips that could be tapped on by users, providing them an easy way to scroll across the whole list of categories. The design of the bar allowed for flexibility in terms of the number of categories, meaning that its layout would remain consistent even if the database would have five or fifty categories. The product listing grid was developed using a responsive two-column layout that displayed product cards in a visually consistent and information-dense format suitable for mobile screens. Each product page includes the product image, name, price, and a favorite button which we put forward the key info a user needs to make a purchase decision without overcluttering the display area. We used lazy load for images which means we only fetch what is currently in the users' view thus reducing initial load time and data use on mobile. We went over touch interaction's quality throughout category and product lists this week. We adjusted tap target sizes to meet accessibility criteria which in turn made all interactable elements large enough for easy tap without need for precise placement of the finger. Scroll behavior was tuned to feel smooth and natural, with appropriate momentum and

deceleration characteristics consistent with native mobile application standards. By providing visual press feedback on which tappable actions have occurred, we have improved the experience of users who may perceive a delay in their taps or touches creating a slight amount of delay between when an action occurs and when they see it occur. The product listing grid utilizes a two-column responsive layout to display all products within a consistent and information condensed way that fits well within the size of most mobile's viewable areas. Each product card displays an image, name, price and toggle button to save a product as a favorite, which provides all necessary information to browse and makes decisions easily but does not take up more space than necessary. Utilizing lazy loading to display images as they enter the viewport improves load time by reducing the amount of data required to load the initial set of products and limits the amount of data consumed on cellular connections. This past week we looked at the quality of touch interactions which we improved across all category and product listing screens. We modified the size of tap targets to meet accessibility criteria which in turn means all interactive elements are large enough to be reliably tapped without requiring precise finger placement. Also we changed the scroll behavior to provide a smooth natural experience with just the right amount of momentum and deceleration which is in line with what is seen in native mobile apps. We introduced visual press feedback to tappable elements which in turn confirms user interactions instantly thus reducing wait time between a tap action and the visible result which in turn minimizes the perception of unresponsive design. By the time we reached the end of the thirteenth week, a completely operational and touch optimized product discovery experience was already set up within the TradeHub application. The filtering mechanism that was set up through categorization, the grid view, as well as the touch interaction design were some of the aspects that served as a good base for developing the product interaction features in the coming weeks.

Practice and Application Activities During 14th Week

In the fourteenth week we saw the roll out of product detail pages as well as the favorites and cart features which in turn saw a large scale expansion of the core shopping function within the TradeHub mobile app. That week we set out to develop a full and reusable product centered experience which took the user from product view through to addition in the cart or favorites at which point we also went in to extend the database schema to support the new features. This week we saw the roll out of the dedicated product detail screen which is what we get each time a user picks a product from within the app from the primary product grid, a category filtered view, a search results page or a recommendation section. We went with a layout that is very logical and which presents info in a very clear visual way to best use the small mobile screen. We put the product image front and center at the top of the screen in a large size which allows the user to really study the product. Below the image we put in the product name, price and a short but informative description which we designed to present the key buy now info at a quick glance. On the product detail page, a quantity selector was added to allow users to determine how many of an item they wish to put into their cart before adding them. The controls for the selector have been clearly marked with increment and decrement buttons which have been made sufficiently large enough for use by touching them. The quantity for this product was set to one since adding zero will mean adding a non-existent item into the cart. The 'add to cart' button can be seen and touched irrespective of where one is on the page. Consequently, the primary purchase action was easily accessible for the user at all times. There are two separate recommendations sections located in the bottom part of the product detail page. The first recommendations section shows people products that are like the one they are viewing but in the same category. The second recommendations section is for things that are frequently purchased together based on the co-purchase data in the database and will help the user find add-on products that other customers bought with the product they are currently using. Each of the recommendation sections is using the same product card component used in other areas of the app for consistent visuals and to ensure that the reusable component works properly across many different screens. One of the major things that we did this week is design a component architecture for our app. Each of the components used in the design of the product details page such as the image component, the title and price component, the quantity picker, the add to cart component, the star component, and the recommendation cards was designed using a modular component design where each of the

component does not depend directly on the other components or on the page itself. The factor is such design make it easier for us to update and reuse throughout the application. This week also involved the new design and addition of the REVIEWS table to the database schema, for a new product review & ratings system. The table design fields included ID which is primary key, USERID referencing the user that created the review, ORDER_ID referencing the order Id through which product was purchased, ITEMID referencing the reviewed Product, RATING contains a numeric score value and COMMENT storing written context of written review text, ISACTIVE supporting soft deletion, CREATEDAT keeps track of timestamp on submission and UPDATEDAT stores timestamp when it was last edited. The ORDER_ID reference field was a technical and intended data integrity choice, so that the system could implement an enforceable validation for a review to be submitted only if there is already a finished order record of the same product by this user. This constraint prevents fraudulent or unverified reviews from being submitted, which is an essential data quality requirement in any production e-commerce platform. The database schema was modified to include the REVIEWS table in order to accommodate product querying. The REVIEWS table contains the following fields: unique identifier (primary key); userID; orderID (tie back to order through which product was purchased); itemID (item being reviewed); rating (numerical score of review); comment (written review); isActive (indicator for soft delete); createdAt (timestamp for when review was submitted); updatedAt (most recent date/time review was last edited). The OrderID field was included as a part of the reference integrity design because this allows for validation that a review can only be added to the database by the user who created the order. This helps to ensure there are no fraudulent or unverifiable reviews in the system, which is a key component of data quality on any e-commerce platform in production. The cart management system has been improved extensively this week as per the needs for the complete workflow before purchase. The controls for quantity management have been provided on the cart itself. This enables the users to add or reduce the quantity of items within the cart directly on the same page, eliminating the need to go to the product details page again. Removal of products from the cart has been provided as an option to allow removal of individual products by simply tapping on them once. A feature to clear the entire cart has also been developed. This allows clearing out the cart instantly without having to delete products one by one. The subtotal, estimated shipping charge, and total have been enabled to automatically update themselves on the order summary tab of the cart upon any change made in the cart.

Practice and Application Activities During 15th Week

During the fifteenth week of development, the TradeHub mobile application focused on building the complete order completion workflow. However, this was one of the most technically difficult parts of the entire mobile development process because the checkout process must be able to combine all elements of authenticating a user, selecting a payment method; creating an order for this user on the server; and allowing this user to confirm their purchase after the process has been completed so that an acceptable transaction experience can be provided. The first task for week fifteen was designing and building out a dedicated secure checkout screen. The purpose of the screen was to guide the user through all of the steps necessary to complete an order prior to confirming their choice, and it was built respecting the most logical order of the steps of the checkout process so that a minimum cognitive load was placed on the user and that all required information was collected and verified before the user completed the checkout process. The screen was divided into separate areas each of which looked at one of the main issues in the check out process which enabled users to work on and change their choices independently of each other and also without one set of fields to influence another set. I put in a great deal of architectural thought into the coordination of client state, server responses, and database records through the check out process. The check out flow which maintained a local state of present address choice, selected payment method, and new card info entered by the user. At the time of order confirmation I would call a server action which included the full check out package that had the present address ID, payment method, and present cart contents which I got right out of the database to avoid any price tampering on the client side. That server action would put together the order in the ORDERS table, each item in the ORDER DETAILS table, put the first note in the ORDER STATUS HISTORY table which included CHANGED BY and CHANGED AT info, and clear the user's cart upon success. I made a complete history in the ORDER STATUS HISTORY table of each step of the order from the first request to delivery, which allowed the customer to see what was happening with their order and gave a good source of information for the admin.

I put a great deal of thought into the architecture to get the various client states, server reports, and database logs to work in sync at each point in the checkout process. Also the local state of the checkout held over the past address choice, payment method and any new card info the user added in at check out. At the time of order submit we'd use the total package (which included address ID, payment method, and present cart -- which we got

out of the database for security reasons) to create the order in the ORDERS table, put in the related line items in the ORDER DETAILS table, made the initial note in the ORDER STATUS HISTORY table and then cleared the user's cart for successful completion. I put in a full history in the ORDER STATUS table of each of the order's status changes from the time it was placed out till delivery which at the same time gave the customer a transparent view of their order's progress and gave us a precise data trail for admin review. I put in edge case handling at every turn to see that the system would stand up to any condition. I had an inline message that came up if the user tried to go forward without a chosen delivery address which told them to pick or add an address first. I also had a server side check that the cart wasn't empty at order submit to stop that. I had error messages for network issues which let the user know of the failure without loss of their check out info so they could try again without reentering anything.

The I designed was the last stage in the checkout process that which the customer would see after placing an order successfully. This screen we put together to present full and very reassuring details of the completed transaction. I displayed the order number which was the main point of reference for the purchase. Also I included a full run down of what was ordered which included product names, quantities, and individual prices which then broke down into the sub total, shipping cost and total amount. Also I showed the which delivery address and payment method were used to get a quick check off of the order details. For customers that paid via bank transfer I included the payment instructions and reference details which they would need to finish that off. At the bottom I put navigation options which took them to the orders section of the account to track the new order which also serves as a transition out.

Practice and Application Activities During 16th Week

Last week continued to be dedicated to the concluding phase of my Work: the final integration and end-to-end testing of the mobile e-commerce app. This is where I combined all of the modules created up to now , and I am reviewing the entire system as a whole, to ensure that all of the components are working correctly when they are tested separately and also when they are working together to create the complete end user experience. This week continued to be dedicated to the concluding phase of my Work: the final integration and end-to-end testing of the mobile e-commerce app. This is where I combined all of the modules created up to now , and I am reviewing the entire system as a whole, to ensure that all of the components are working correctly when they are tested separately and also when they are working together to create the complete end user experience. Also, I conducted an intensive analysis of the entire architecture of the application, analyzing the way each mobile screen connects to the backend via secure operations on the server side. As a result, it turned out that no confidential information could be displayed on the client side, and any interaction with the database is done following the pattern repository, just as in the case of the web version of the application.

This week I spent a lot of time dealing with edge cases and boundary tests. Some of them are moving forward with an empty cart, trying to finalize the order but without saved delivery address, inputting invalid payment details, and network failures during the checkout process. In those edge cases, I added user-friendly messages for users to prevent the application from crashing or showing errors. Also, I checked the UI for consistency across different screen sizes and ran the app on multiple device configurations to ensure that layouts, fonts, and interactive elements were correct on each device. I fixed the slight spacing and alignment discrepancies that I found at this stage to maintain a clean and professional look throughout the app. The lessons that I have learned through the final build and test process of my internship include some important fundamental aspects of developing applications for mobile phones like the importance of defensive programming, performing comprehensive testing of boundaries, and proper segregation of front-end state management and back-end data manipulation. My completion of a mobile application that is ready for deployment has served as proof of the overall value of all technical skills I have gained throughout the internship.

CHAPTER 3: INSTRUCTIONS RECEIVING AND GIVING PROCESS

3.1 Instructions Receiving Process

Throughout the internship, all instructions were received verbally from Kadir Kalkan, a company partner and co-owner of Adım Yazılım. Feedback and progress reviews were conducted on a bi-weekly basis, allowing me to adjust my work according to the expectations of the organization at regular intervals.

3.1.1 Instructions Receiving Task 1 – Database Design

During my first 1.5 months of internship I was involved in design of the MS SQL Server database schema for TradeHub, an online shopping platform that I was given over by Kadir Kalkan. At that time the main focus of the instructions was on the what of the e commerce platform and the key entities that should be present in the database structure. We had bi weekly meetings which reported my progress. In these meetings Kadir Kalkan would review the database structure and give in to me his input on table relationships, normalization, indexing strategies, and data integrity constraints. At that stage the greatest challenge we had was when we had many inter connected tables and we had to maintain referential integrity between them at the same time we had to see to it that our schema was scalable and maintainable. I worked on the feedback given out during review meetings and improved my skills as I went along to present a in detail Entity Relationship Diagram that included the main entities like USERS, ORDERS, ORDERDETAILS, ADDRESS, and ITEMS. The supervisor has looked at the database schema (Figure 45 and Figure 47) and gave index structures and table relationships his feedback in person.

3.1.2 Instructions Receiving Task 2 – Web Frontend Development

Upon the conclusion of the database development process, I was then asked by Kadir Kalkan to start working on the frontend side of the TradeHub application via Next.js. The entire process took about five to six weeks, with weekly bi-weekly meeting reviews.

These discussions revolved around page structure and component design, alongside integrating the frontend with the backend that utilized SQL Server through the repository pattern approach. However, one of the challenges experienced at this point included creating server-side rendering without running into hydration problems, as well as securing database details. Through the use of feedback from review meetings and our official docs I was able to sort out those issues. At the end of this phase we had put together a fully

functional web front end which included product lists, product info pages, shopping cart features, favorite products, check out options and an admin panel.

During this phase instructions were given out verbally by Kadir Kalkan which mostly revolved around the development of the web front end of the platform using Next.js. Also as reported in Figure 46 it was decided at this stage that we would go ahead and develop the web and mobile platforms as one single project. Throughout this phase all instructions and progress reports were discussed via bi weekly meetings.

3.1.3 Instructions Receiving Task 3 – Mobile Application Development

Upon finishing the database development phase, Kadir Kalkan ordered me to start working on the web-based frontend of the TradeHub platform using Next.js. It took around five to six weeks for this phase including regular bi-weekly review meetings.

These meetings were mainly used to talk about the arrangement of pages, design of components, and frontend integration with the SQL Server backend via the repository pattern. One of the biggest problems during this stage was the implementation of server-side rendering with the Next.js App Router without causing hydration issues and exposing the database connection details. Through the use of feedback obtained during review meetings and referring to the documentation, I have been able to solve all problems. At the completion of this phase, I have built a fully working web front-end that includes functionalities like product listing, details pages, shopping cart, favorites, checkout, and administration functionalities. The instructions for this phase were provided verbally by Kadir Kalkan and involved building the web frontend of the application utilizing Next.js. As can be seen from Figure 46, it was agreed during this phase that both web and mobile platforms should be built as part of a single project structure.

CHAPTER 4: DISCUSSION AND CONCLUSION

Adım Yazılım's intern program which BTK Akademi hosted was a game changer for my academic and professional growth. It was by far the most rewarding and practical based experience of my university years. In the internship I got to know the basics and in depth skills of database design and management, full stack programming, and mobile app development. The software I finally put out was a fully functional TradeHub e-commerce platform which included database, web, and mobile front ends.

MS SQL Server was the choice of software in the second phase of the internship, where I needed to design the database for the TradeHub platform. It was a monumental task as I had to build the entire relational diagram right down to USERS, ORDERS, ORDERDETAILS, ITEMS, ADDRESS, PAYMENTS, INVOICE, and INVOICEDetail, the major transactional tables, as well as secondary tables like FAVORITES, REVIEWS, USERCARDS, and ORDER_STATUS_HISTORY, which support the operation of various other features. Normalization, the right choice of data types, primary and foreign key constraints, and enforcement of referential integrity were among the considerations taken into account when designing the tables. The aim was to have a schema that could reliably and consistently handle the implementation of the full feature set of the application. But the coursework did not end with designing tables. Advanced database programming such as stored procedures, triggers, indexing strategies, transaction management, and backup and recovery exercises was carried out. My working with these features in a targeted manner has enhanced my knowledge of enterprise-level database systems and their administration and protection and that, in turn, served as a very solid technical base for all the other development work that followed. Secondly, there was the implementation of the front end of the TradeHub platform based on the use of the Next.js App Router framework. In this stage, I was able to get practical skills on how server-side rendering works, how repository-based data fetching is carried out, and also how to create a scalable component structure. Some of the features that were implemented during this stage included: the ability to list products with sorting based on categories and searching, dynamic product details pages with recommendations sections, the ability to have favorite products stored persistently for easy retrieval, the fully functional shopping cart with the capacity to manage quantities and perform order calculations, a secure checkout process where authentications are enforced as well as server-side prices calculations, and an admin dashboard with functionalities such as order and product

management by roles. The integration of the frontend components together with the backend running SQL server with the help of the repository pattern gave me practical experience in developing full-blown database-driven web applications where separation of concerns is clearly achieved with regard to data access layer, business logic and presentation layer. Reflecting on the entire internship what we did which scaled up a platform brought to light some key issues. We identified that very early care put into the database schema before we jumped into front end development was the most important. That issue came in at third until we saw that changes which we made in the first phase without proper thought actually reduced our data fetch options later on in the web and mobile app stages. Also we saw that use of the same architectural patterns like a shared repository layer between front ends became a great support as the code base grew complex and as we thought, as we had at first which was that new features would be easy, they in fact became a major issue in that we introduced inconsistencies or logic duplication. We put in a lot of effort into thinking through edge cases and we ended up going back to the drawing board many times during the web and mobile beta testing phases when we came up with user scenarios we hadn't thought of in which the app didn't function as we had planned let alone run smoothly. That internship gave me a deep insight into how software development professionals work in a real world organization. I had Kadir Kalkan giving us bi weekly feedback which was like our check ins as a team and I learned to take criticism well and use it to improve the project. Also I saw the value of code management, writing documentation and differentiating between the various elements of the application architecture as we worked with a large scale of code over many development phases.

REFERENCES

- [1] Adım Yazılım, "SQL Server Training Content." Adım Yazılım. [Online]. Available: <https://www.adimyazilim.com.tr/> [Accessed: May 2026].
- [2] Microsoft, "SQL Server Official Documentation." Microsoft. [Online]. Available: <https://www.microsoft.com/en-us/sql-server/> [Accessed: May 2026].
- [3] Microsoft, "Transact-SQL Reference." Microsoft Docs. [Online]. Available: <https://learn.microsoft.com/en-us/sql/t-sql/language-reference> [Accessed: May 2026].
- [4] Vercel, "Next.js Official Documentation." Vercel. [Online]. Available: <https://nextjs.org/> [Accessed: May 2026].
- [5] React Native, "React Native Official Documentation." Meta. [Online]. Available: <https://reactnative.dev/> [Accessed: May 2026].
- [6] BTK Akademi, "Course Materials." BTK Akademi. [Online]. Available: <https://www.btkakademi.gov.tr/> [Accessed: May 2026].
- [7] Flutter Documentation, "Flutter Documentation." Flutter. [Online]. Available: <https://docs.flutter.dev/> [Accessed: May 2026].

APPENDIX

Appendix A – Week 1: SQL Server Setup & System Databases

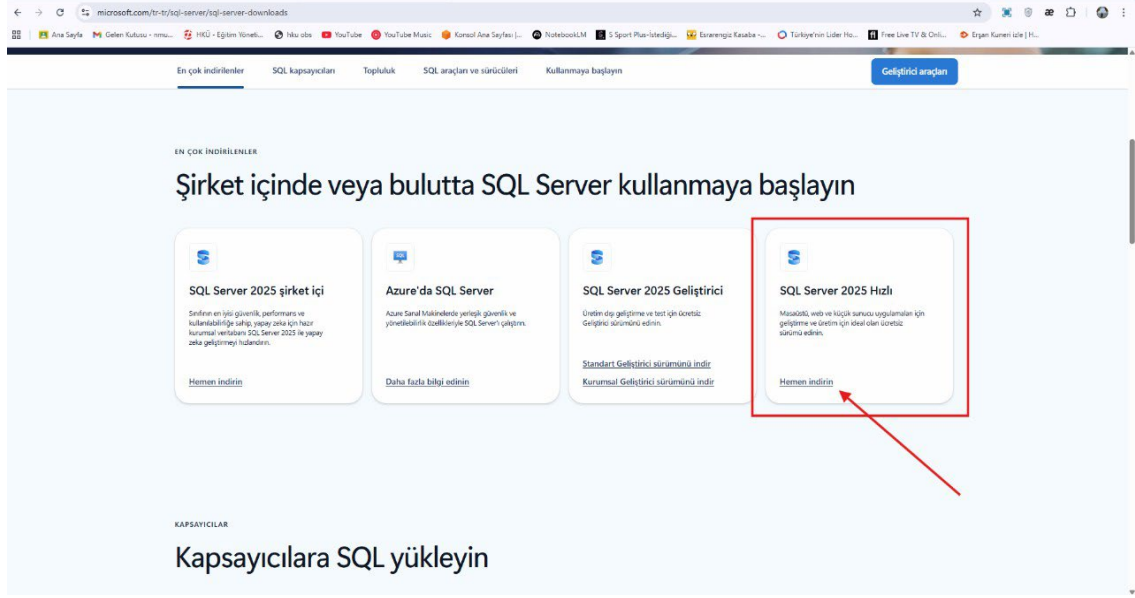


Figure 1 Mssql installation screenshot

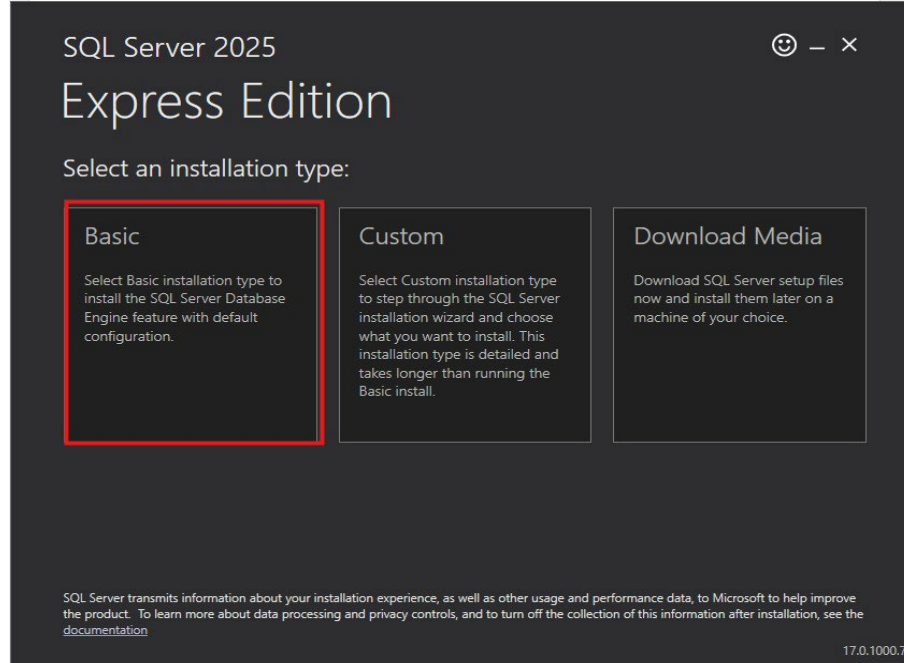


Figure 2 Windows OS setup screenshot

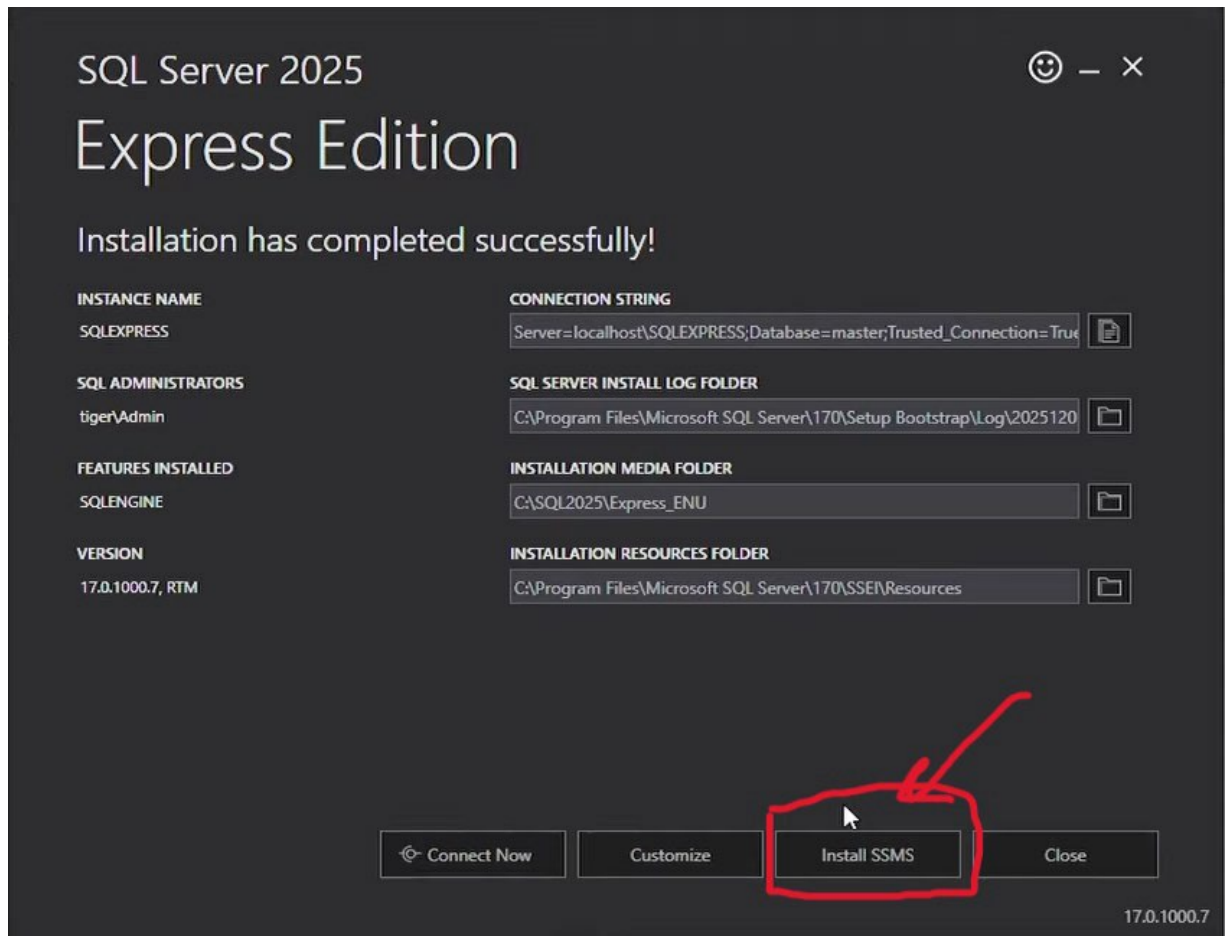


Figure 3 SQL Server installation screenshot

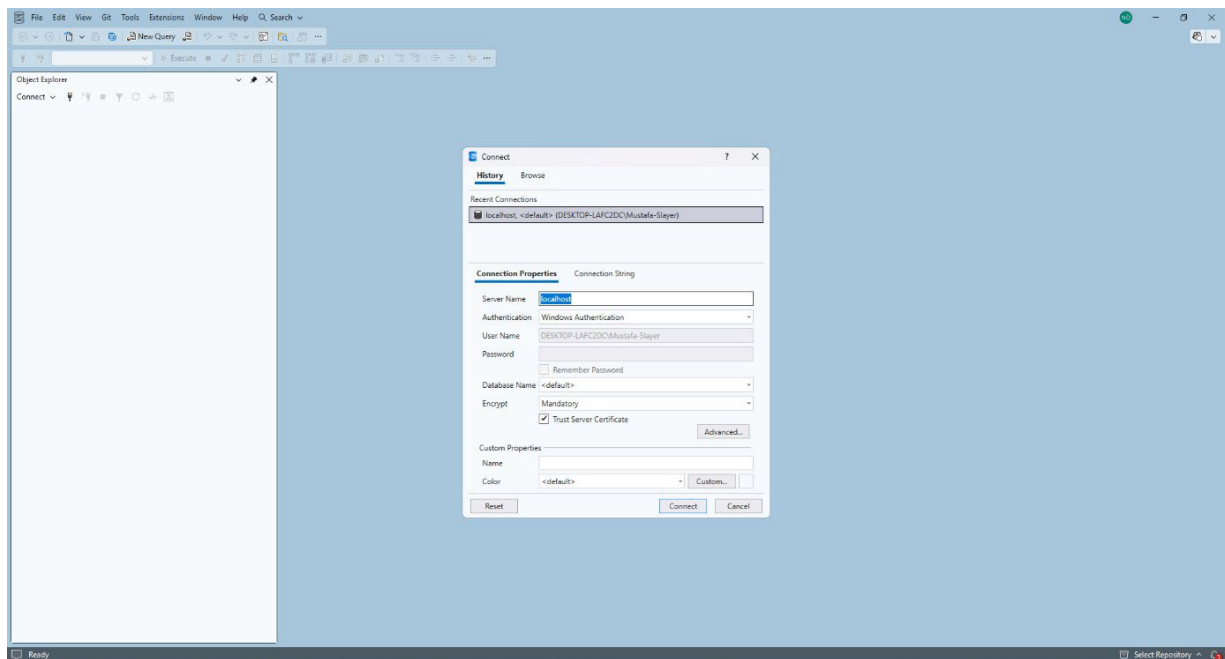


Figure 4 SSMS launch screenshot

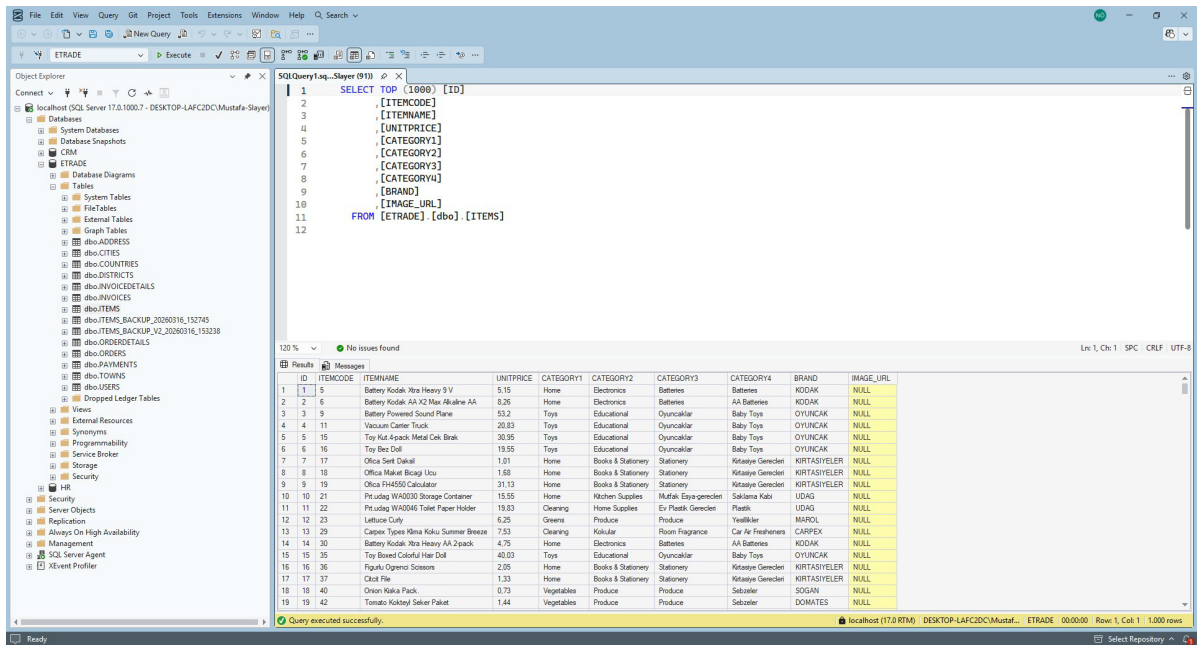


Figure 5 Sample database creation screenshot

Appendix B – Week 2: T-SQL Fundamentals & Table Design

ORDERS		
ID	ID	INT IDENTITY(1,1)
USERID	KULLANICI ID	INT
DATE_	TARİH SAAT	DATETIME
TOTALPRICE	TOPLAM TUTAR	FLOAT
STATUS_	DURUM	TINYINT
ADDRESSID	ADRES ID	INT
ORDERDETAILS		
ID	ID	INT IDENTITY(1,1)
ORDERID	SİPARİŞ ID	INT
ITEMID	ÜRÜN ID	INT
AMOUNT	MİKTAR	INT
UNITPRICE	BİRİM FİYAT	FLOAT
LINETOTAL	SATIR TOPLAMI	FLOAT

Figure 6 Orders and OrderDetails Foreign Key Relationship

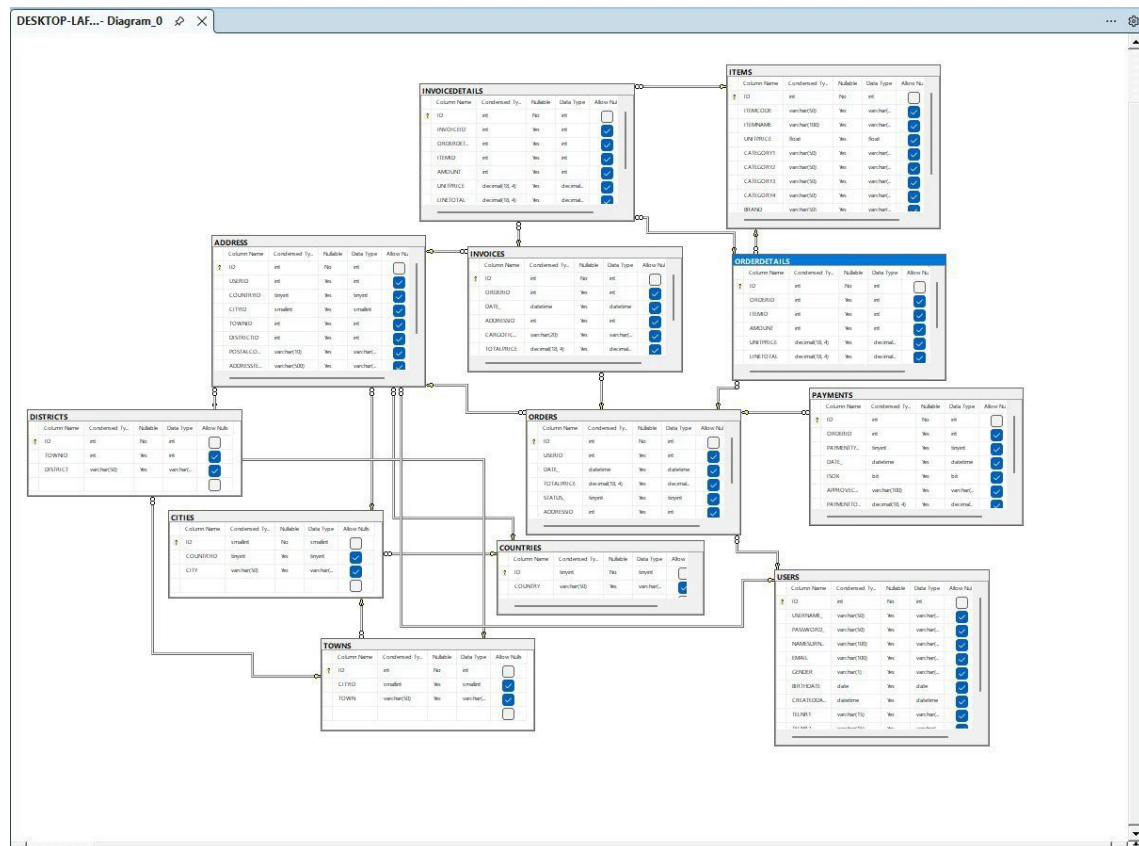


Figure 7 Normalization Overview (Diagram)

Appendix C – Week 3: Advanced SQL & Indexing

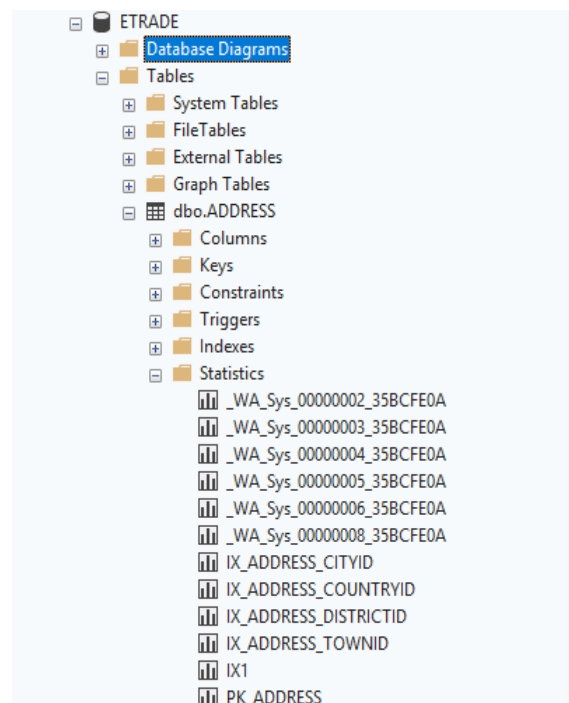


Figure 8 Clustered vs Non-Clustered Index test results

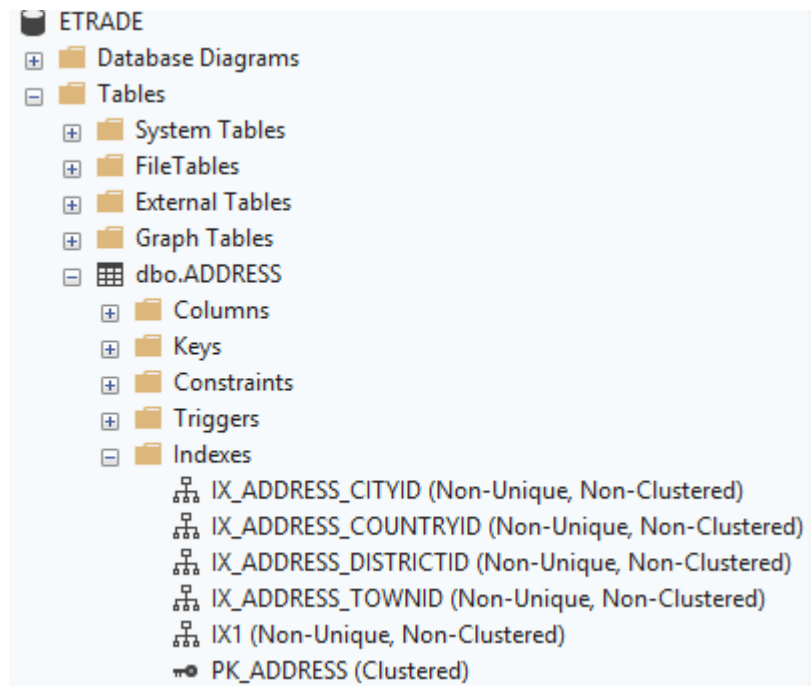


Figure 9 Clustered vs Non-Clustered Index

Appendix D – Week 4: Transaction Management, Backup and Recovery

SQLQuery1.s...Slayer (63))

1 SELECT TOP 5 * FROM USERS;

120 % No issues found Ln: 1, Ch: 27 (26 chars, 1 lines) TABS CRLF UTF-8

ID	USERNAME	PASSWORD	NAMESURNAME	EMAIL	GENDER	BIRTHDATE	CREATEDDATE	TELNR1	TELNR2
1	N_OZSIMITCI	doropipa	Nadican OZSIMITCI	N_OZSIMITCI@sqlegitimbi.com	K	1951-12-29	2018-07-25 01:35:35.000	(555)3852463	(532)5898448
2	E_SELIM	xpaia	Emihan SELIM	E_SELIM@sqlegitimbi.com	E	1993-01-21	2018-07-28 01:35:35.000	(543)8894233	(534)8242021
3	S_VLGEN	4042521	Soner ULGEN	S_VLGEN@sqlegitimbi.com	E	1940-12-30	2018-07-29 01:35:35.000	(534)9377724	(544)1214495
4	C_BORKLV	audraia	Çetin BORKLÜ	C_BORKLV@sqlegitimbi.com	E	1946-09-28	2018-07-30 01:35:35.000	(563)7748268	(505)9661278
5	E_IBUKVRTVNCV	transatlantic	Ezgi IBUKÜRTÜNÇÜ	E_IBUKVRTVNCV@sqlegitimbi.com	K	1985-02-16	2018-07-31 01:35:36.000	(537)7721023	(554)1253351

Figure 10 Initial state of the USERS table before transaction


```

1 BEGIN TRANSACTION;
2
3 INSERT INTO USERS
4 VALUES (
5     'TEST_USER',
6     'test123',
7     'Test Kullanıcı',
8     'test@test.com',
9     'E',
10    '2000-01-01',
11    GETDATE(),
12    '(555)1111111',
13    '(555)2222222'
14 );
15
16 SELECT TOP 5 * FROM USERS ORDER BY 1 DESC;
17
18 ROLLBACK;
19
20 SELECT TOP 5 * FROM USERS ORDER BY 1 DESC;

```

ID	USERNAME	PASSWORD	NAMESURNAME	EMAIL	GENDER	BIRTHDATE	CREATEDDATE	TELNR1	TELNR2
9681	TEST_USER	test123	Test Kullanıcı	test@test.com	E	2000-01-01	2026-04-14 15:03:10.313	(555)1111111	(555)2222222
9680	Mustafa	password	Mustafa Özkaya	mustafa-1453@hotmail.com	NULL	NULL	2026-03-16 13:58:15.273	NULL	NULL
9679	test_user	12345	Test User	test@example.com	NULL	NULL	2026-03-12 23:37:13.560	NULL	NULL
9678	S_KOSKA	kaför	Şeyma KOSKA	S_KOSKA@sqlegitimbtbk.com	K	1989-07-23	2017-02-28 01:52:58.000	(538)2724261	(541)9996357
9677	O_SAYMAZLAR	r3a0m1sa1n	Onur SAYMAZLAR	O_SAYMAZLAR@sqlegitimbtbk.com	E	1995-05-24	2017-02-27 01:52:58.000	(542)2259981	(543)2648916

Figure 11 Transaction rollback demonstration in the USERS table

Appendix E – Week 5: Next.js UI/UX Edge Cases

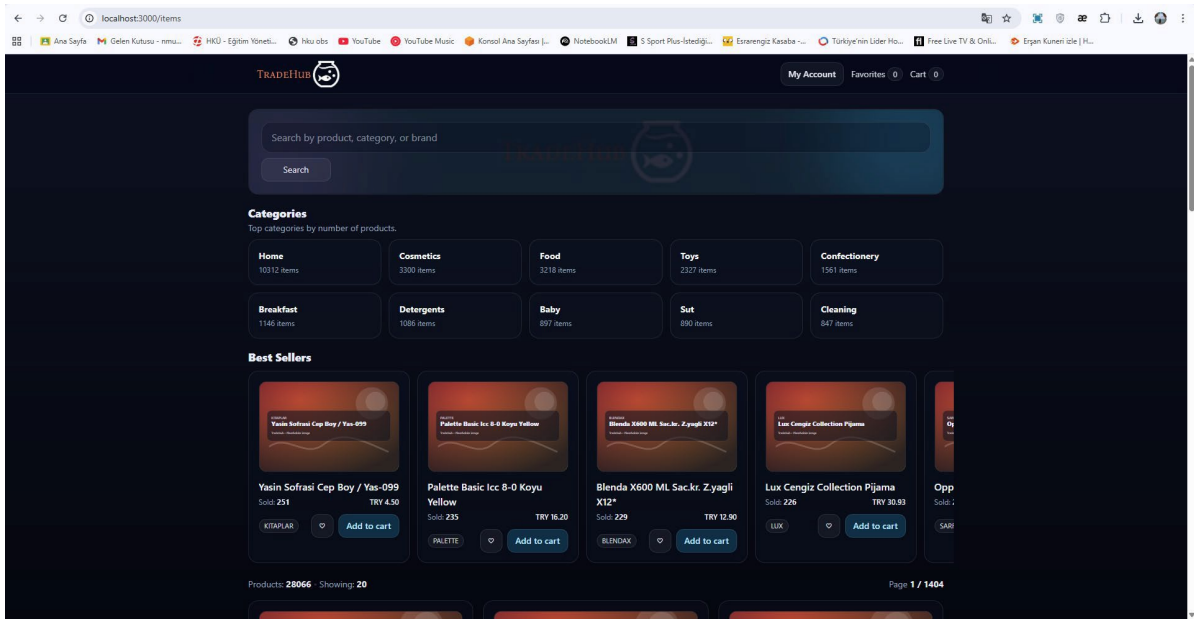


Figure 12 Product Listing Page with Lazy-Loaded Images (UI Screenshot)

```

page.tsx x layout.tsx
src > app > items > page.tsx > ItemsPage
16 export default async function ItemsPage({
42   const showing = result.items.length;
43
44   return (
45     <div className="space-y-4">
46       <section className="relative overflow-hidden rounded-2xl border border-white/10 bg-slate-900/25 p-5">
47         <div className="pointer-events-none absolute inset-0 opacity-[0.12]">
48           <Image
49             src="/TradeHub-horizontal.png"
50             alt=""
51             fill
52             priority
53             sizes="(max-width: 1200px) 100vw, 1200px"
54             className="object-cover object-center blur-[1px]"
55           />
56         </div>
57         <div className="pointer-events-none absolute -right-24 -top-24 h-72 w-72 rounded-full bg-sky-400/20 blur-3xl" />
58         <div className="pointer-events-none absolute -bottom-24 -left-24 h-72 w-72 rounded-full bg-fuchsia-400/10 blur-3xl" />
59         <div className="relative">
60           <form action="/items" method="get" className="flex w-full flex-col gap-2">
61             <input
62               name="q"
63               defaultValue={q}
64               placeholder="Search by product, category, or brand"
65               className="w-full rounded-2xl border border-white/10 bg-slate-950/30 px-4 py-3 text-base outline-none focus:border-white/20"
66             />
67             <div className="flex flex-col gap-2 md:flex-row">
68               <input type="hidden" name="category" value={category} />
69               <button className="rounded-xl border border-white/10 bg-white/5 px-3 py-2 text-sm hover:bg-white/10 md:w-28">
70                 Search
71               </button>
72             </div>
73           </form>
74         </div>
75       </section>
76     </div>
77   )

```

Figure 13 Lazy Loading Implementation (Relevant Code Snippet)

Appendix F – Week 6: Favorites System & Cross-Page Consistency

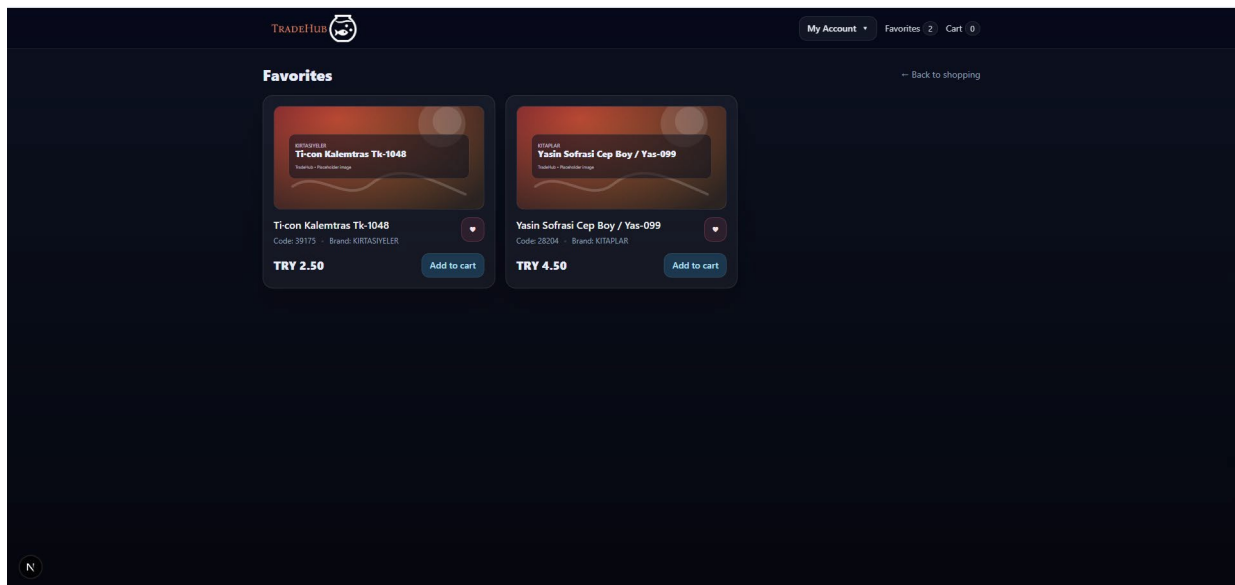


Figure 14 Favorites on product listing page

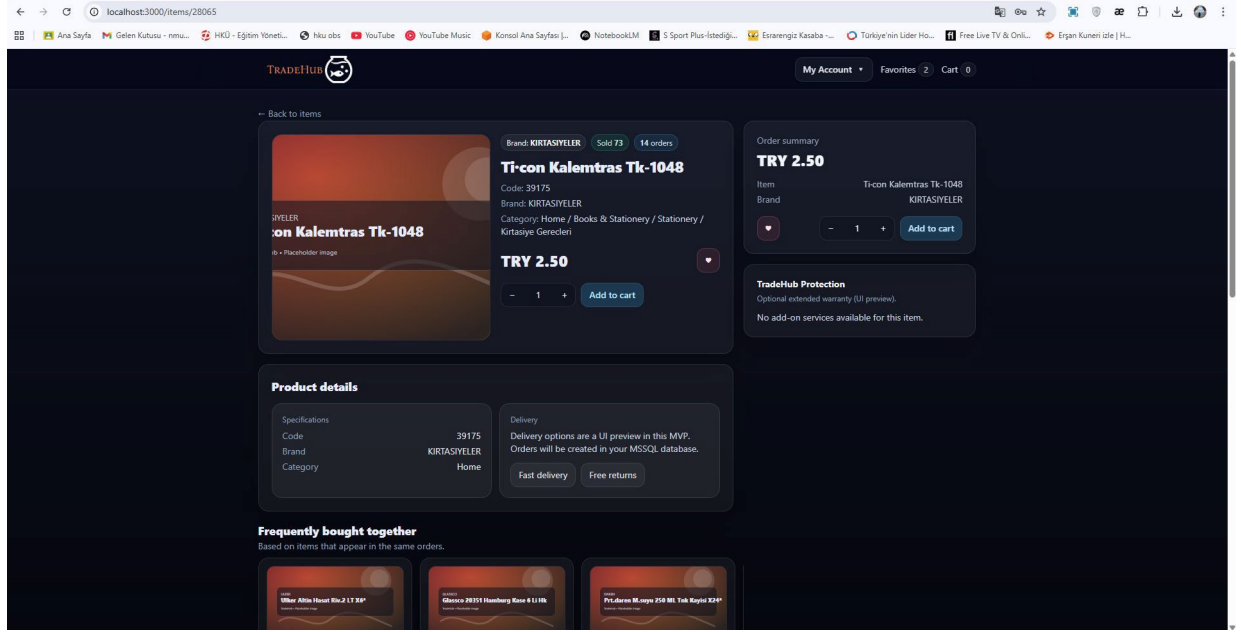


Figure 15 Product detail page

Appendix G – Week 7: Checkout Flow

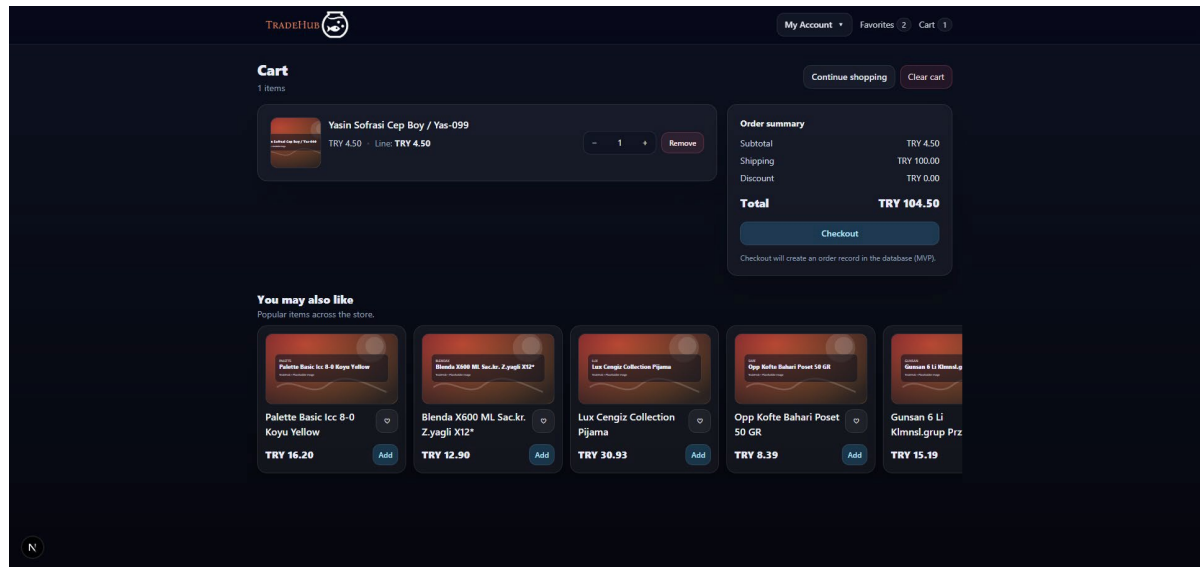


Figure 16 Cart UI

Appendix H – Week 8: Full Workflow & Integration

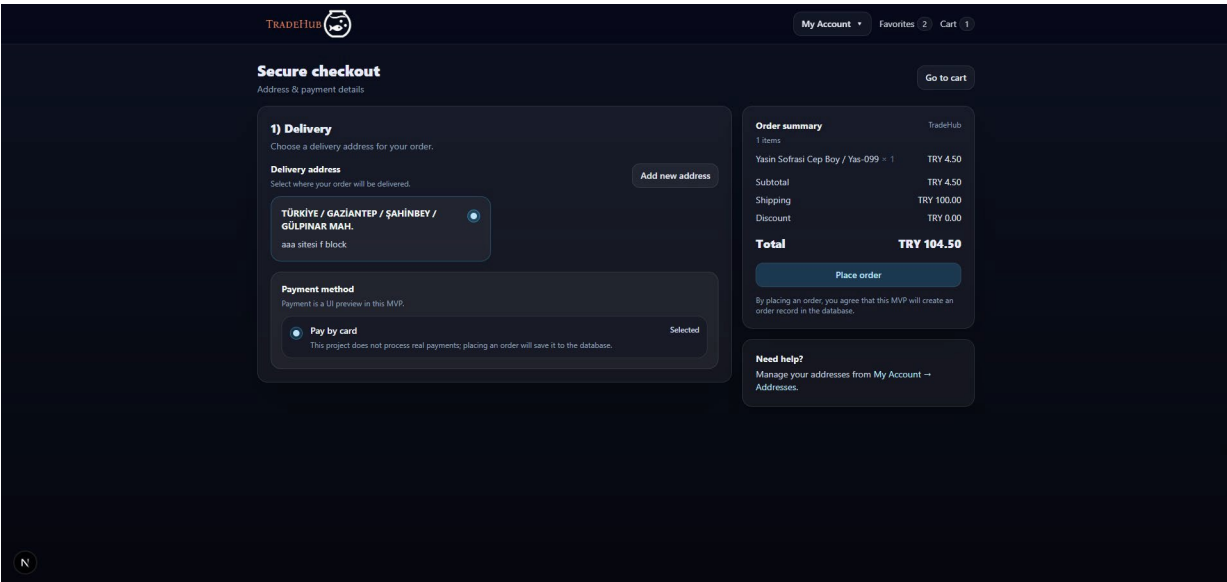


Figure 17 Checkout form with addresses

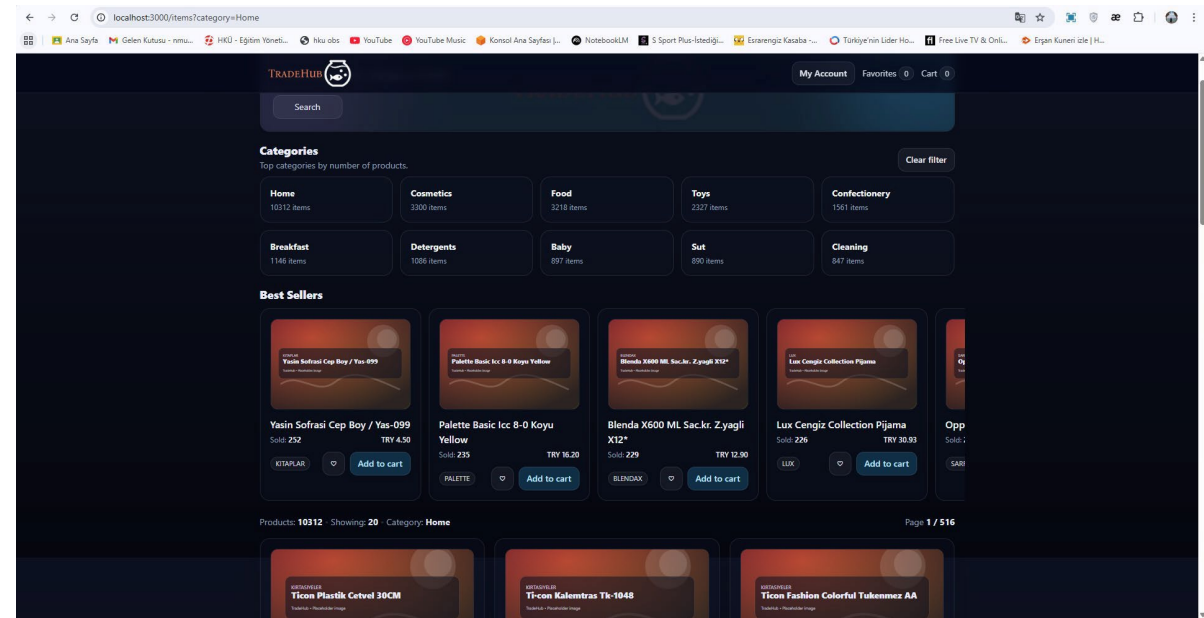


Figure 18 Search by Categories

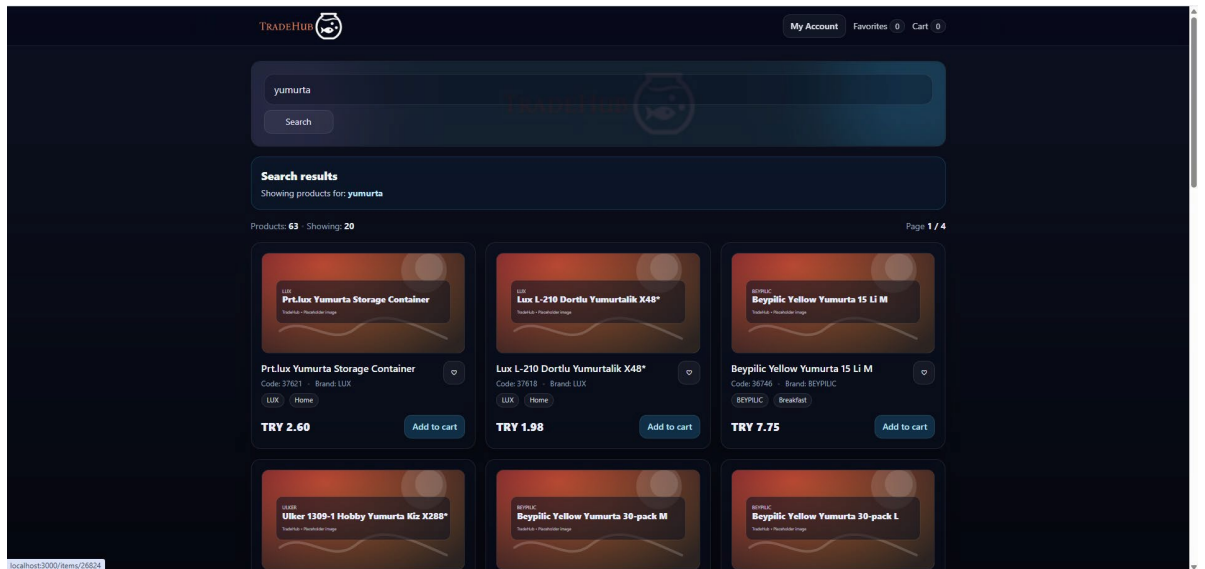


Figure 19 Showing products for Search

Appendix I – Week 9: Main, Account and Address Pages

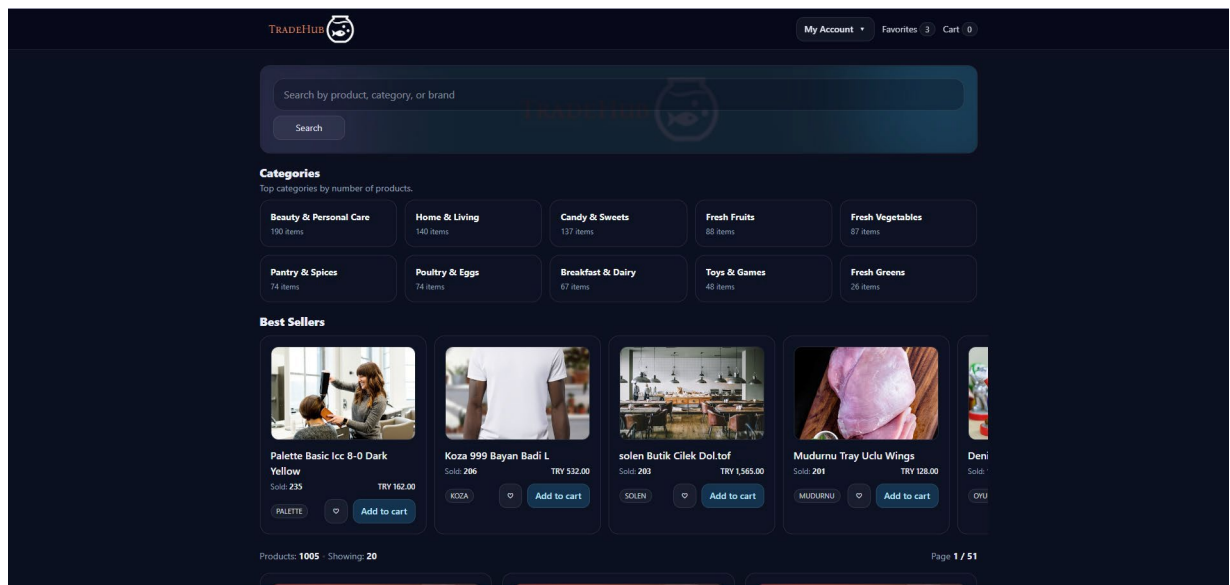


Figure 20 Main Page

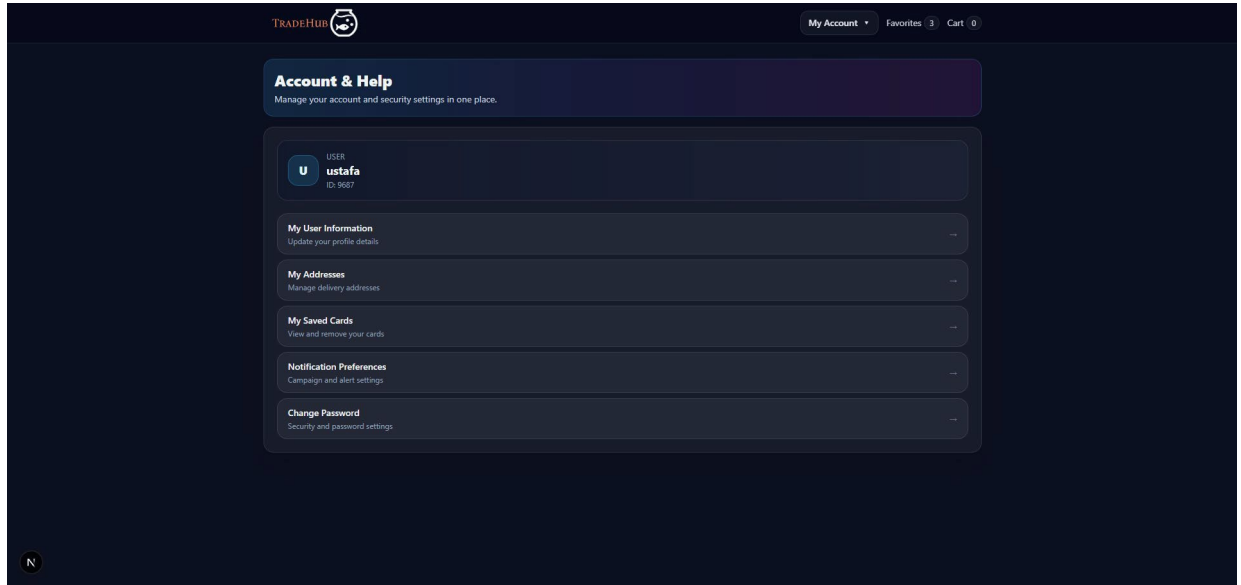


Figure 21 Account Page

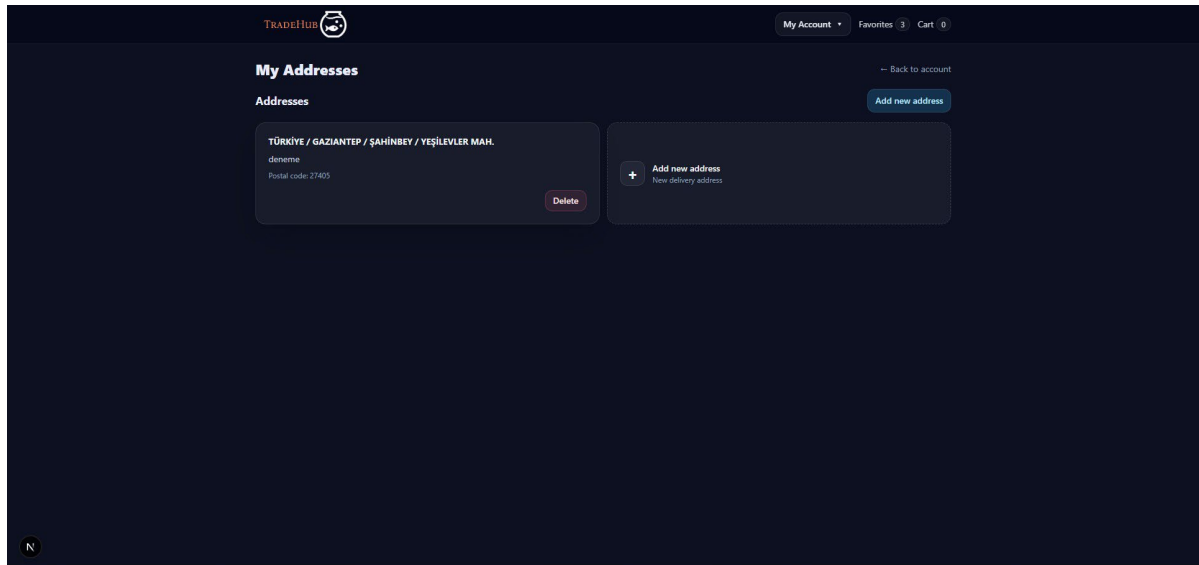


Figure 22 Addresses Page

Appendix J – Week 10: Product Detail, My Orders and Cart Flow

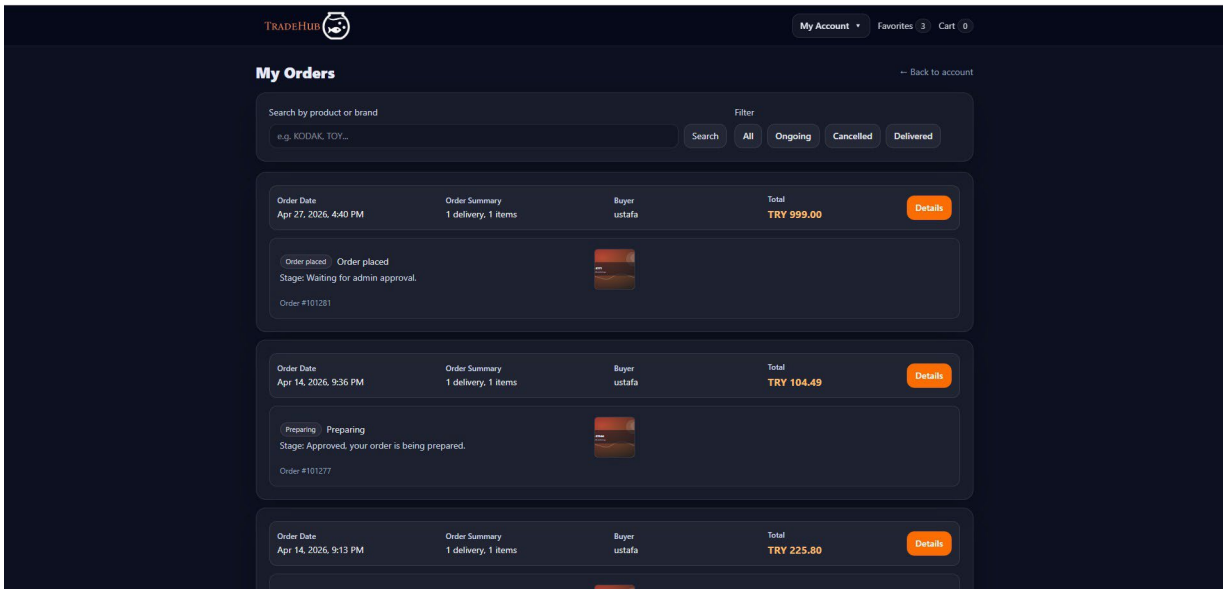


Figure 23 Order Page

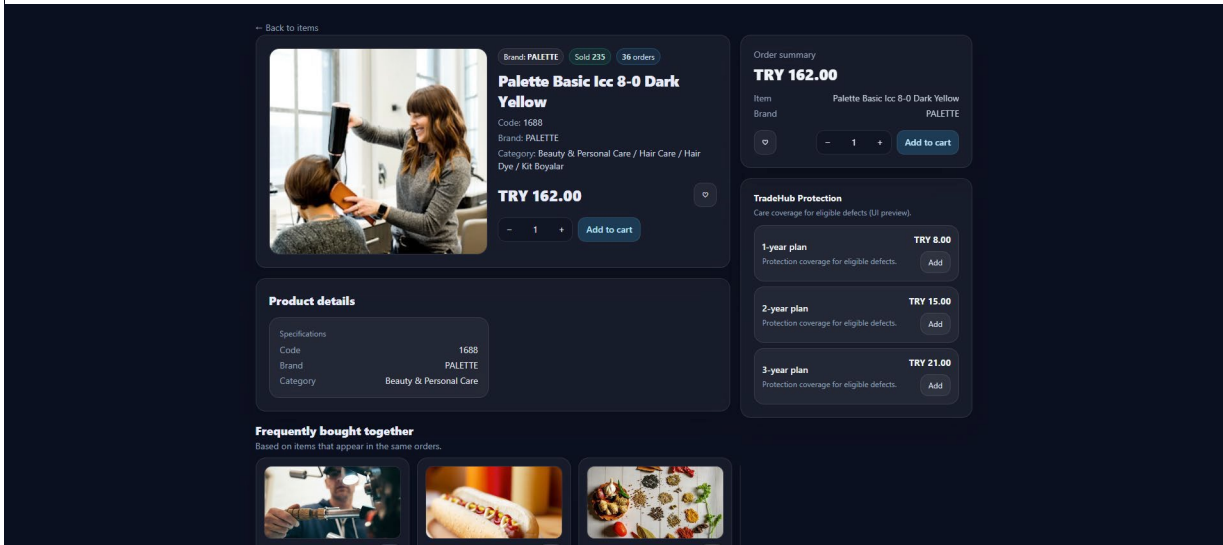


Figure 24 Product Details Page

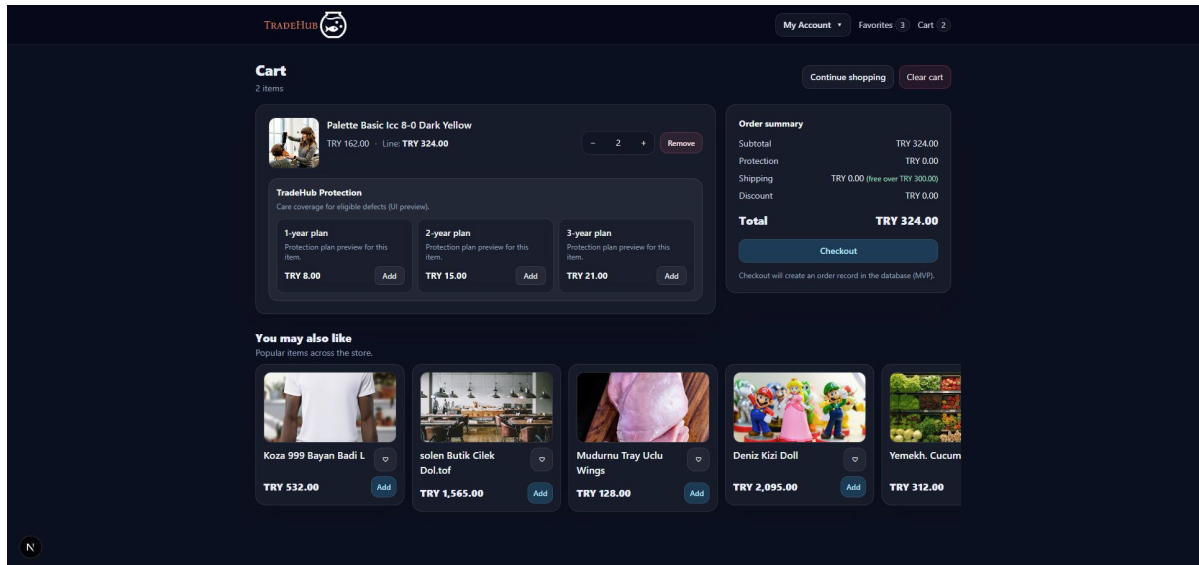


Figure 25 Cart Page

Appendix K – Week 11: Secure Checkout and Order Completion

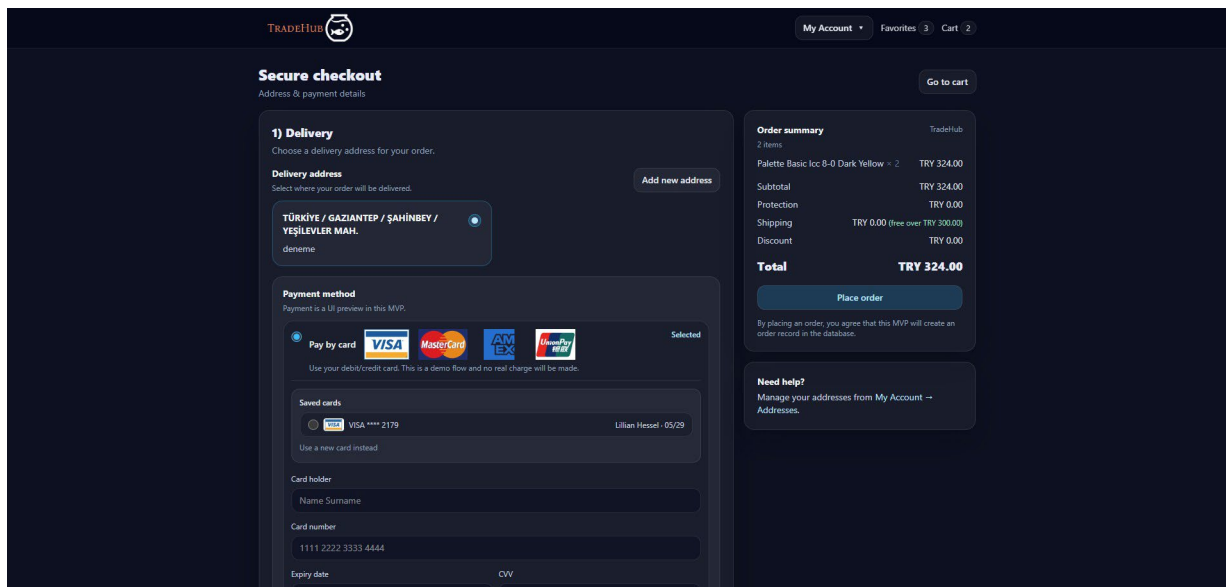


Figure 26 Secure Checkout Page

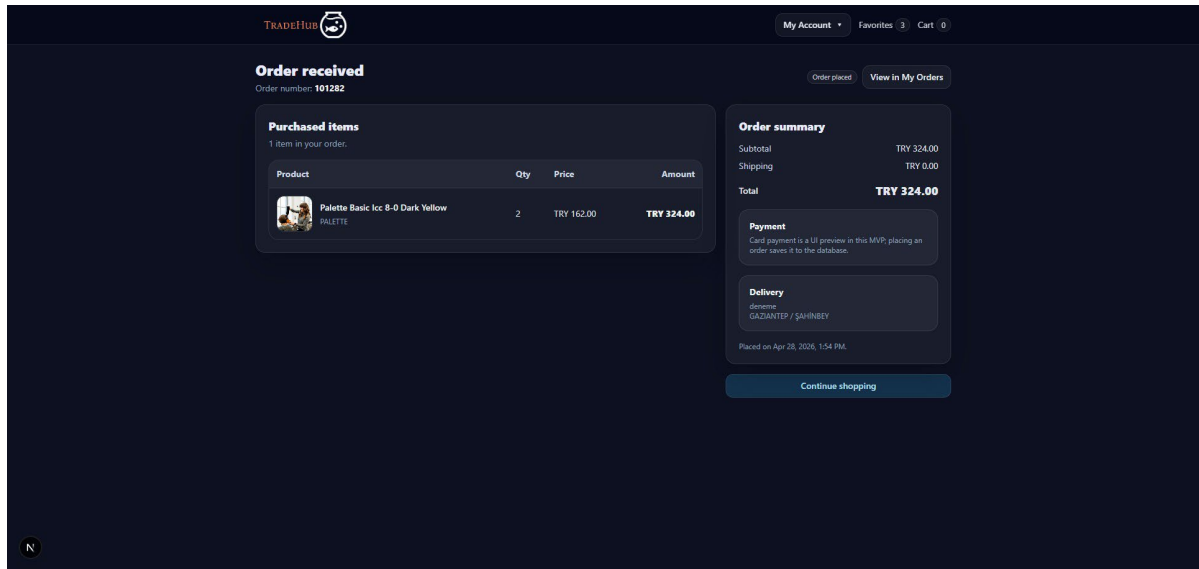


Figure 27 Order Received Page

Appendix L – Week 12: Admin Panel

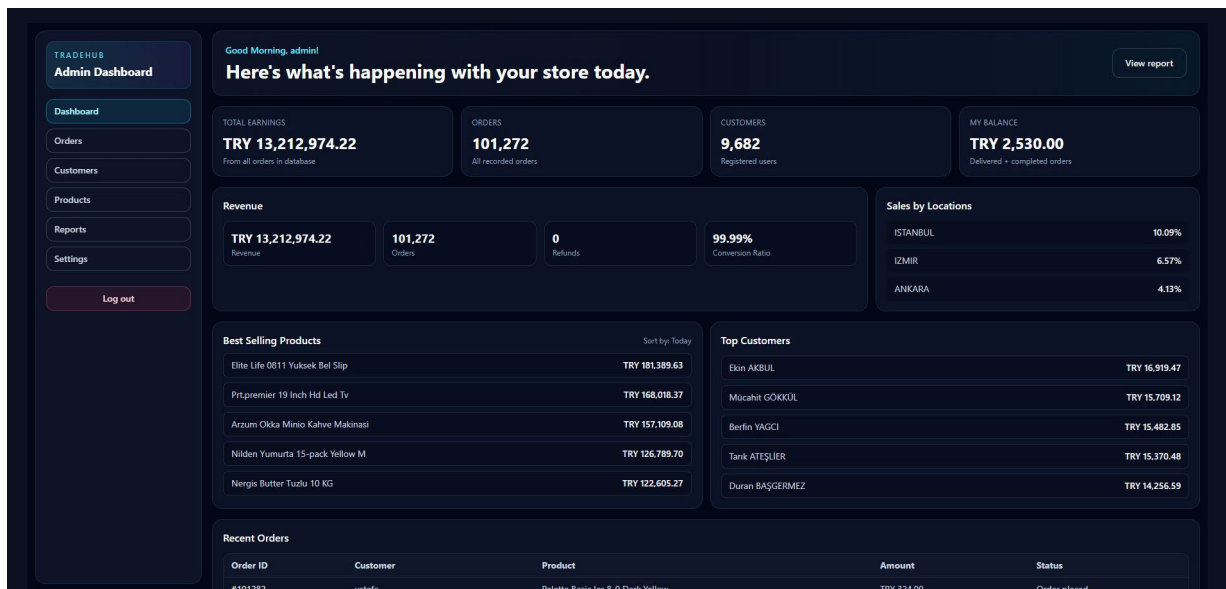


Figure 28 Admin Dashboard

ADMIN Orders Back to dashboard				
Order	Customer	Status	Total	Action
#101282	ustafa	Order placed	TRY 324.00	Details
#101281	ustafa	Order placed	TRY 999.00	Details
#101280	yenikayit	Completed	TRY 1.998.00	Details
#101279	Mustafa Özkaya	Shipped	TRY 2.629.00	Details
#101278	Mustafa Özkaya	Completed	TRY 532.00	Details
#101277	ustafa	Preparing	TRY 104.49	Details
#101276	ustafa	Shipped	TRY 225.80	Details
#101275	ustafa	Shipped	TRY 179.29	Details
#101274	ustafa	Order placed	TRY 174.89	Details
#101273	ustafa	Order placed	TRY 183.78	Details
#101272	Mustafa Özkaya	Order placed	TRY 104.50	Details
#101271	Mustafa Özkaya	Order placed	TRY 108.26	Details
#101270	Test User	Order placed	TRY 10.30	Details
#74354	İlhan UZUNDAG	Order placed	TRY 0.00	Details
#19550	Gökçe YASA	Shipped	TRY 153.85	Details
#80073	Yusuf Ali BEKTAŞER	Order placed	TRY 0.00	Details
#30947	Serdar GÖKBURUN	Order placed	TRY 0.00	Details

Figure 29 Admin Orders

Order Details ← All Orders

Order No
#101282

Order Date
28.04.2026 13:54:18

Order Summary
1 package, 1 items

Order Status
Order placed, waiting admin approval.

Total
TRY 324.00

Invoice

Order placed

Order placed, waiting admin approval.

ORDER PROGRESS

1

Order placed

2

Preparing

3

Shipped

4

Delivered

5

Completed

Admin Actions

Approve

Report

Order Lines



Palette Basic Icon 8-0 Dark Yellow

PALLETTE

Qty: 2, Unit: TRY 162.00 - Amount: TRY 324.00

Delivery Address

ustafa
deneme
GAZIANTEP / ŞAHİNBEY

Invoice Address

ustafa
deneme
GAZIANTEP / ŞAHİNBEY

Payment Info

Subtotal

TRY 324.00

Shipping

TRY 0.00

Total

TRY 324.00

Figure 30 Admin Order Details

Appendix M – Week 13: Mobile Product Discovery and Categories

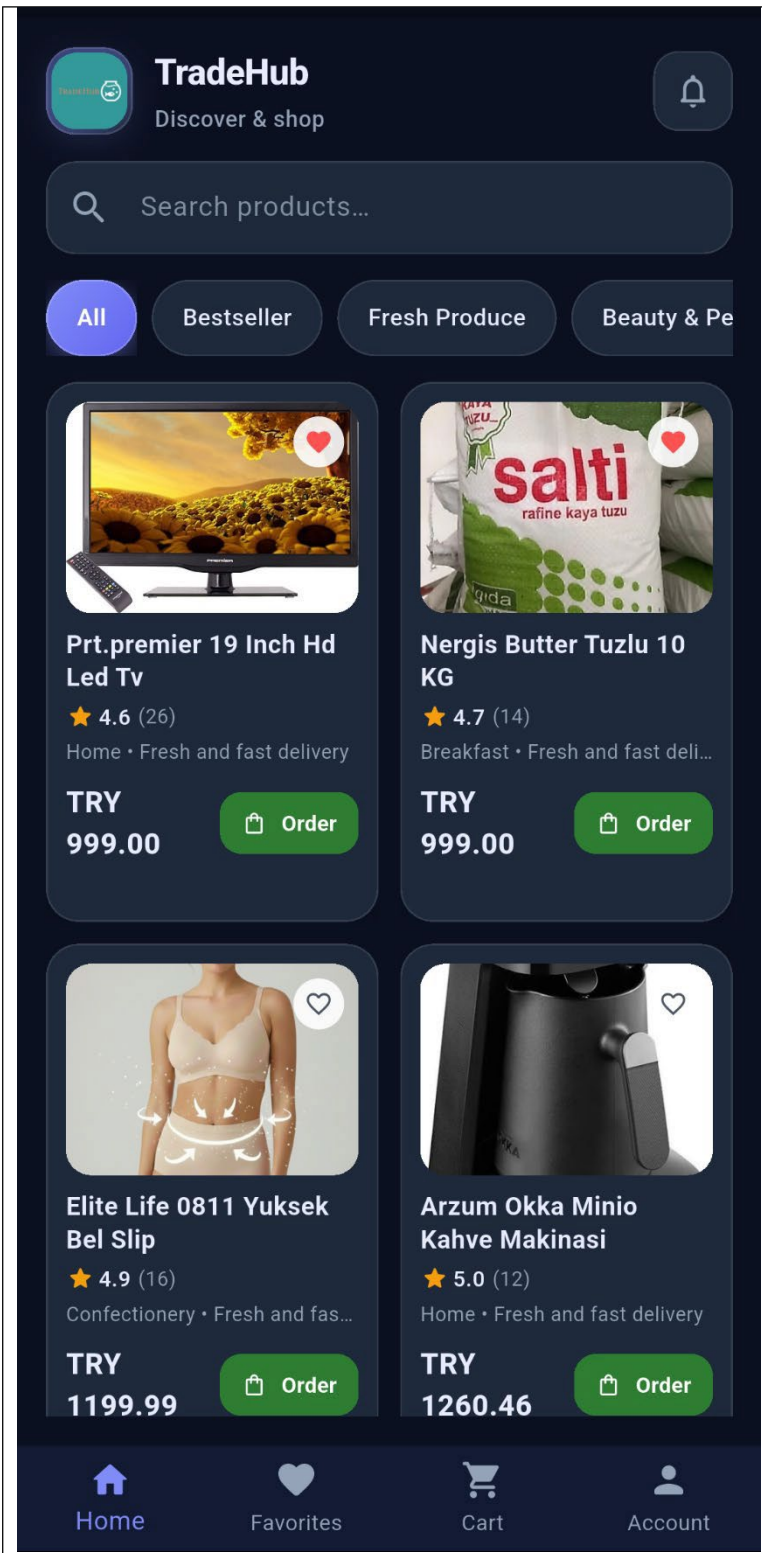


Figure 32 Screenshot of the category strip

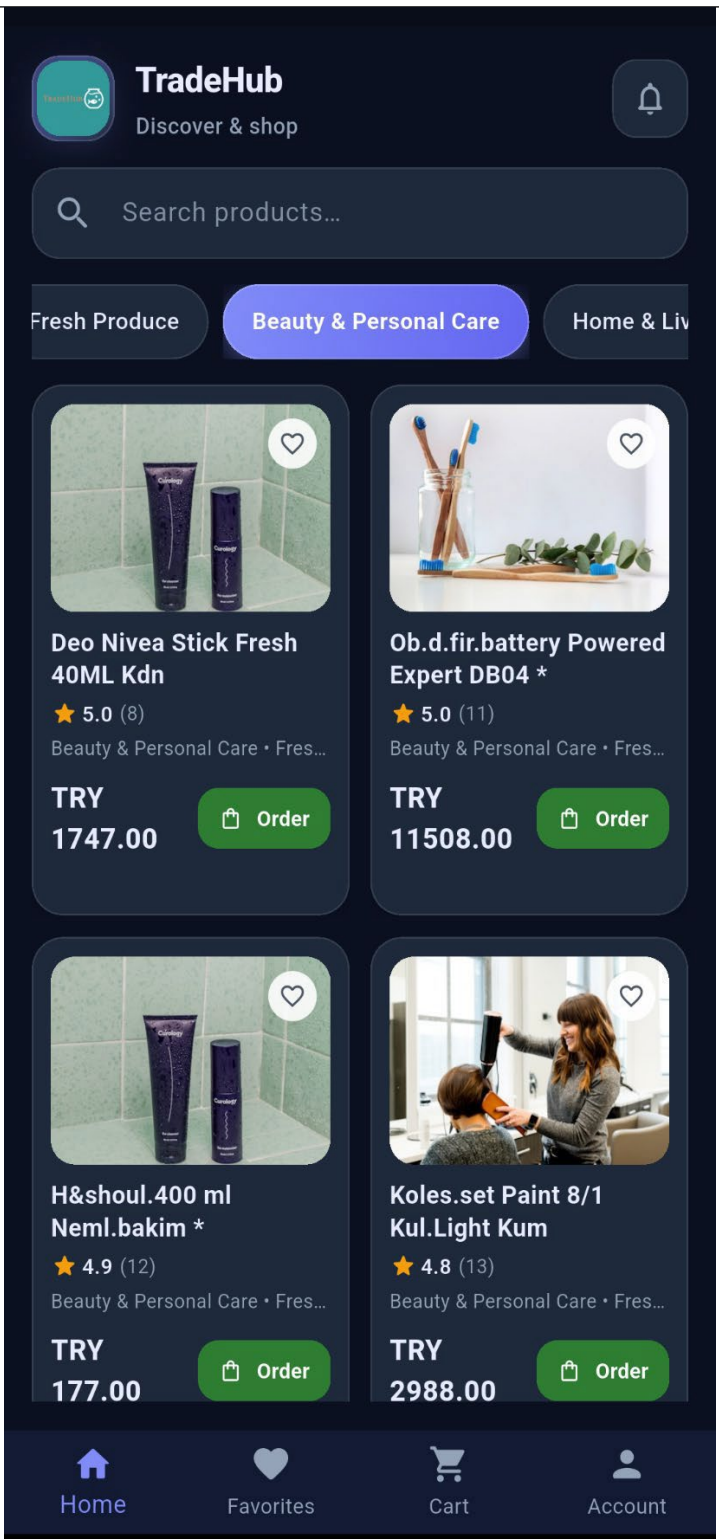


Figure 31 Product list filtered by selected category

Appendix N – Week 14: Mobile Product Detail and Favorites

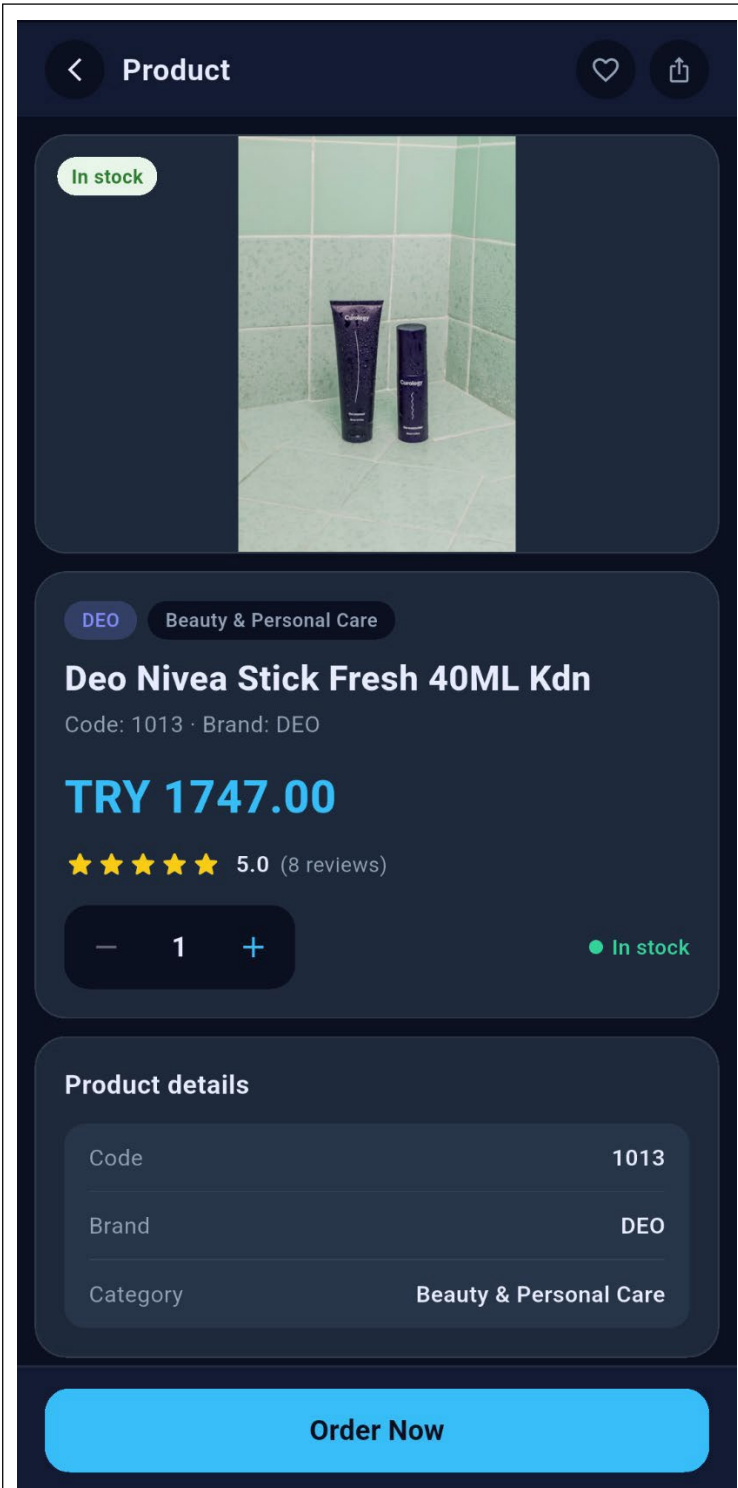


Figure 33 Product detail page screenshot



Figure 34 Favorites screen

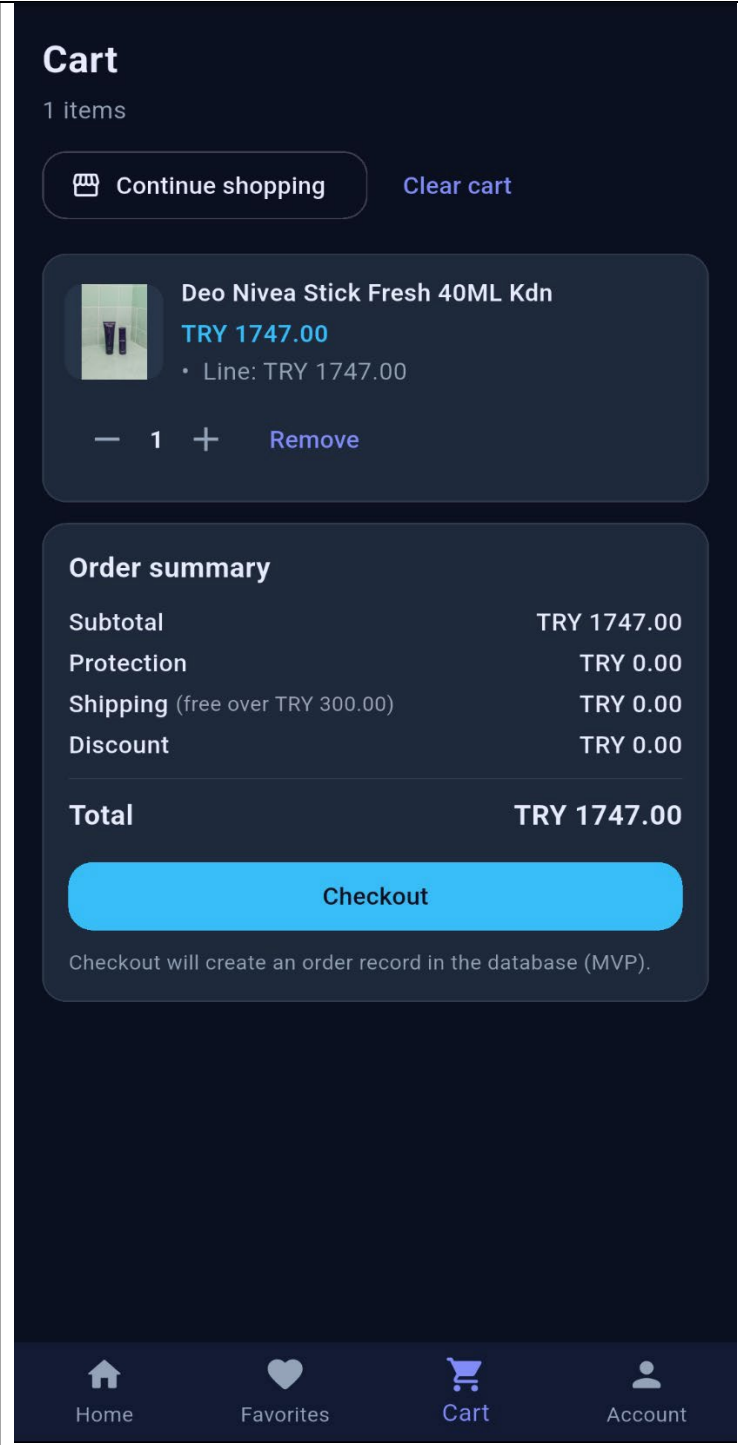


Figure 36 Cart screen

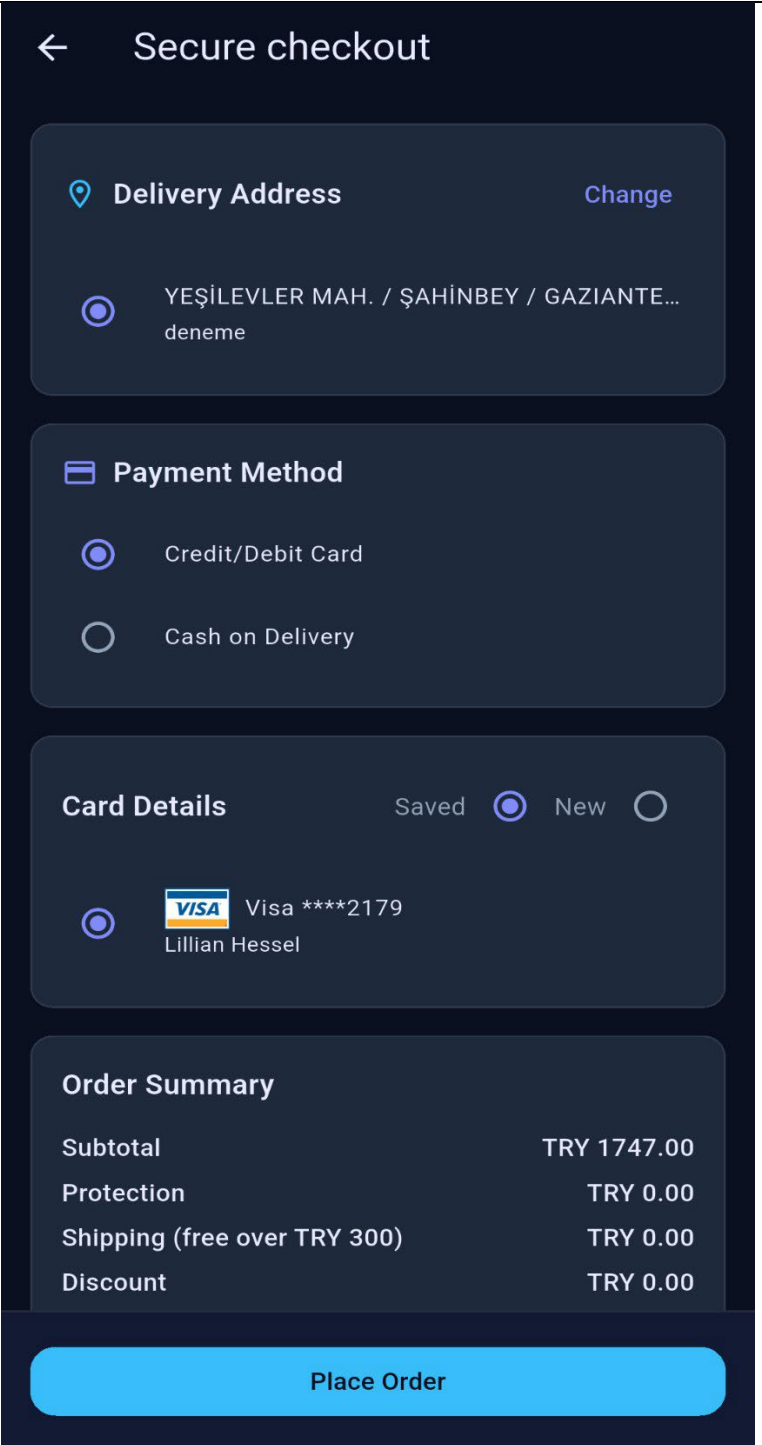


Figure 35 Secure Checkout screen

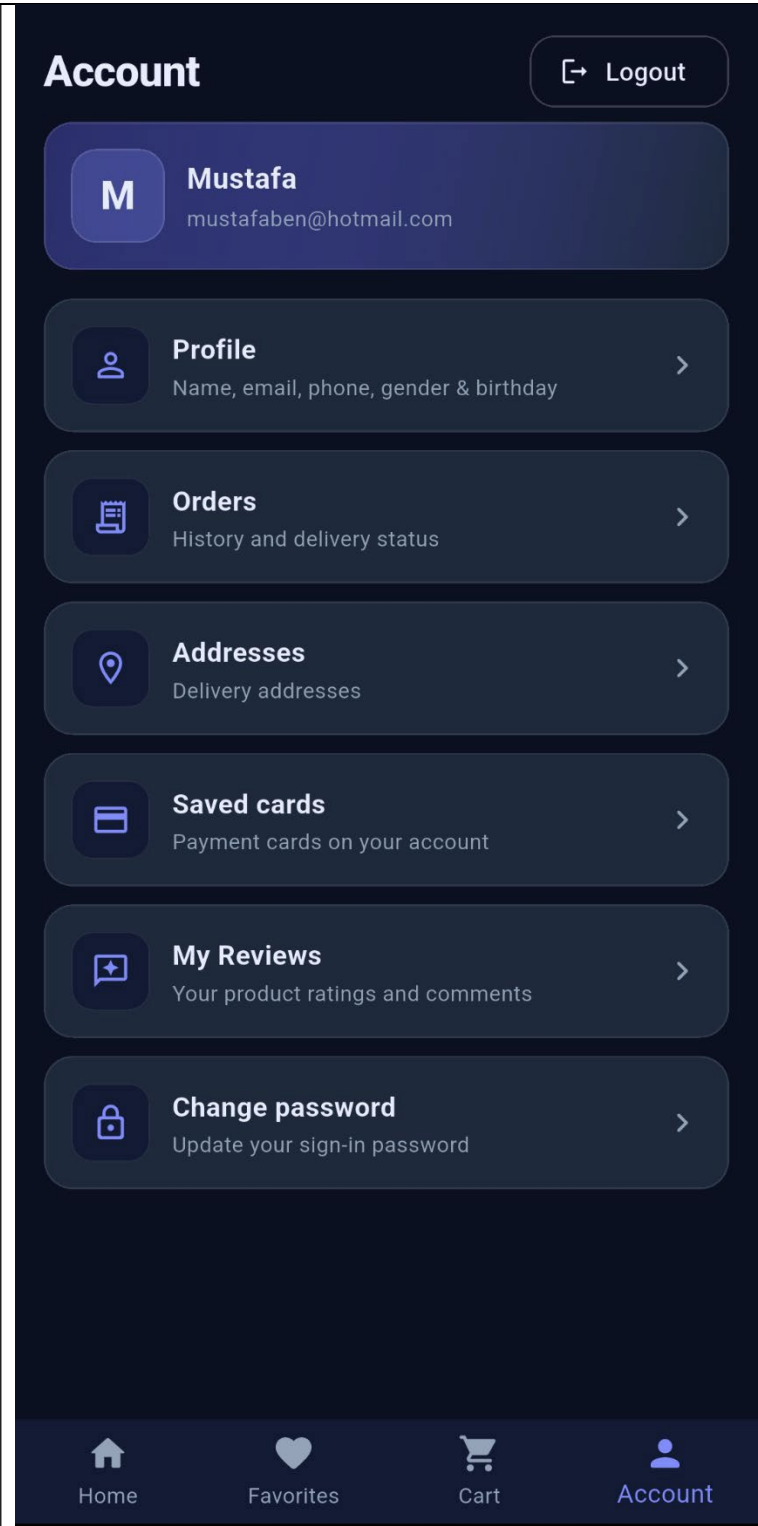


Figure 38 Account Page

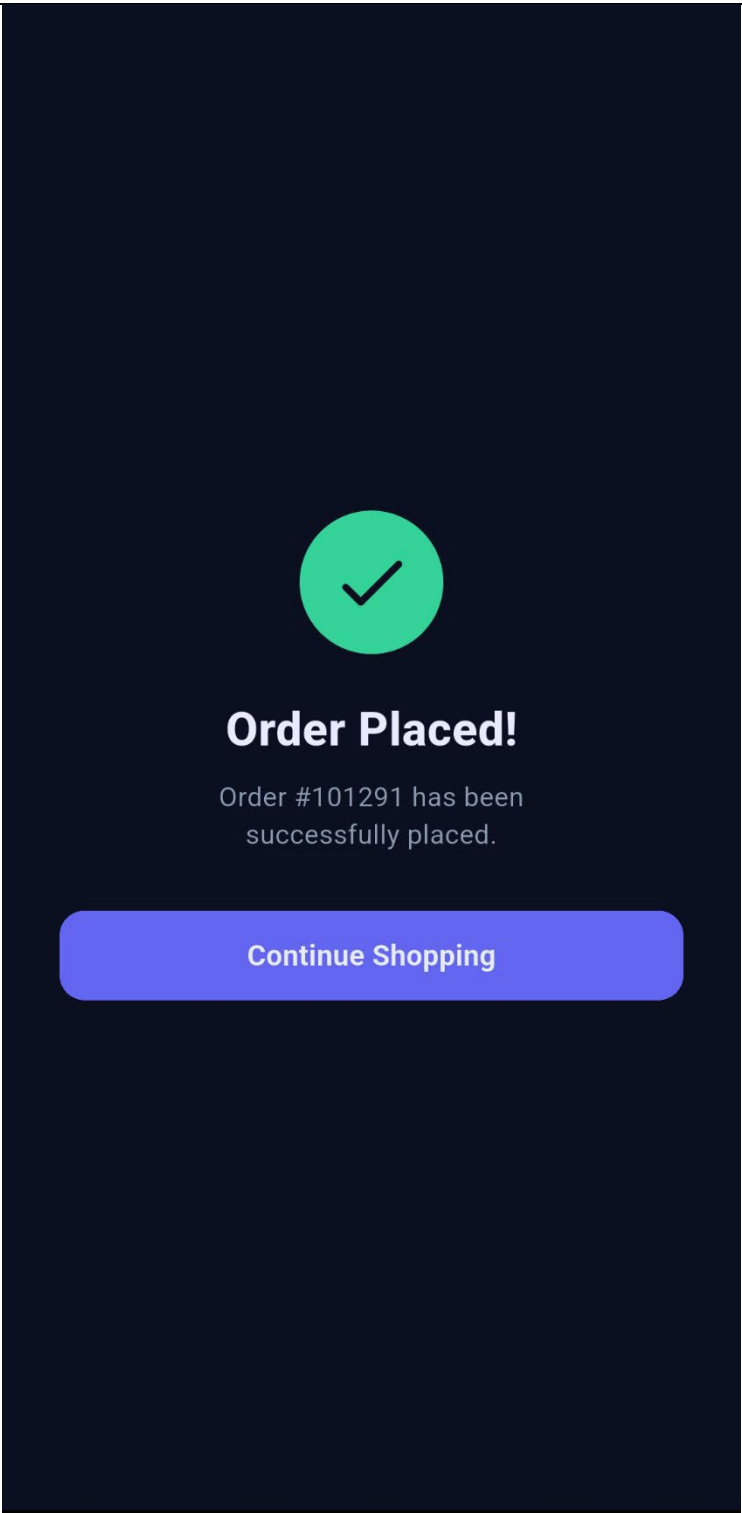


Figure 37 Order Successfully Screen

Appendix P – Week 16: Mobile Final Integration and End-to-End Testing

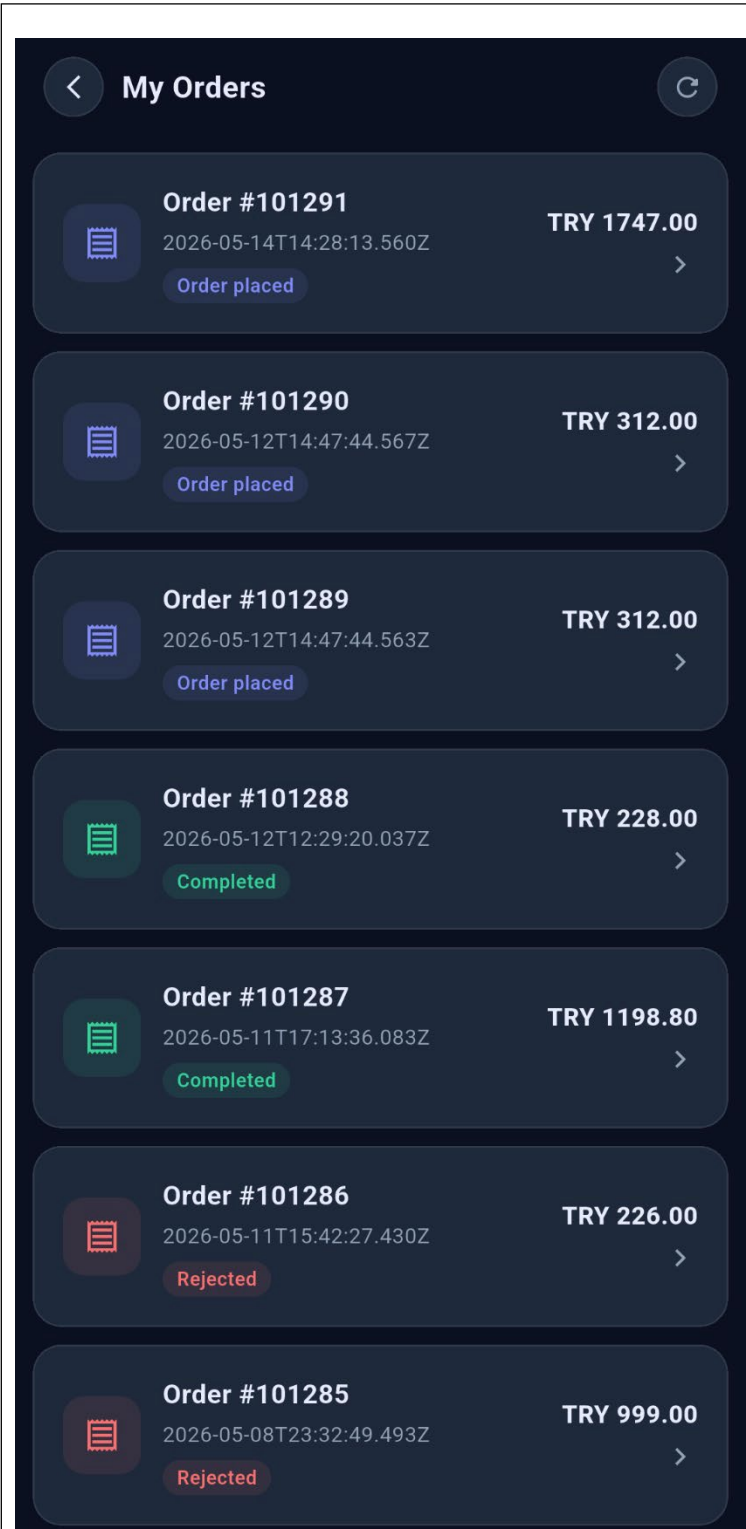


Figure 40 Orders Page

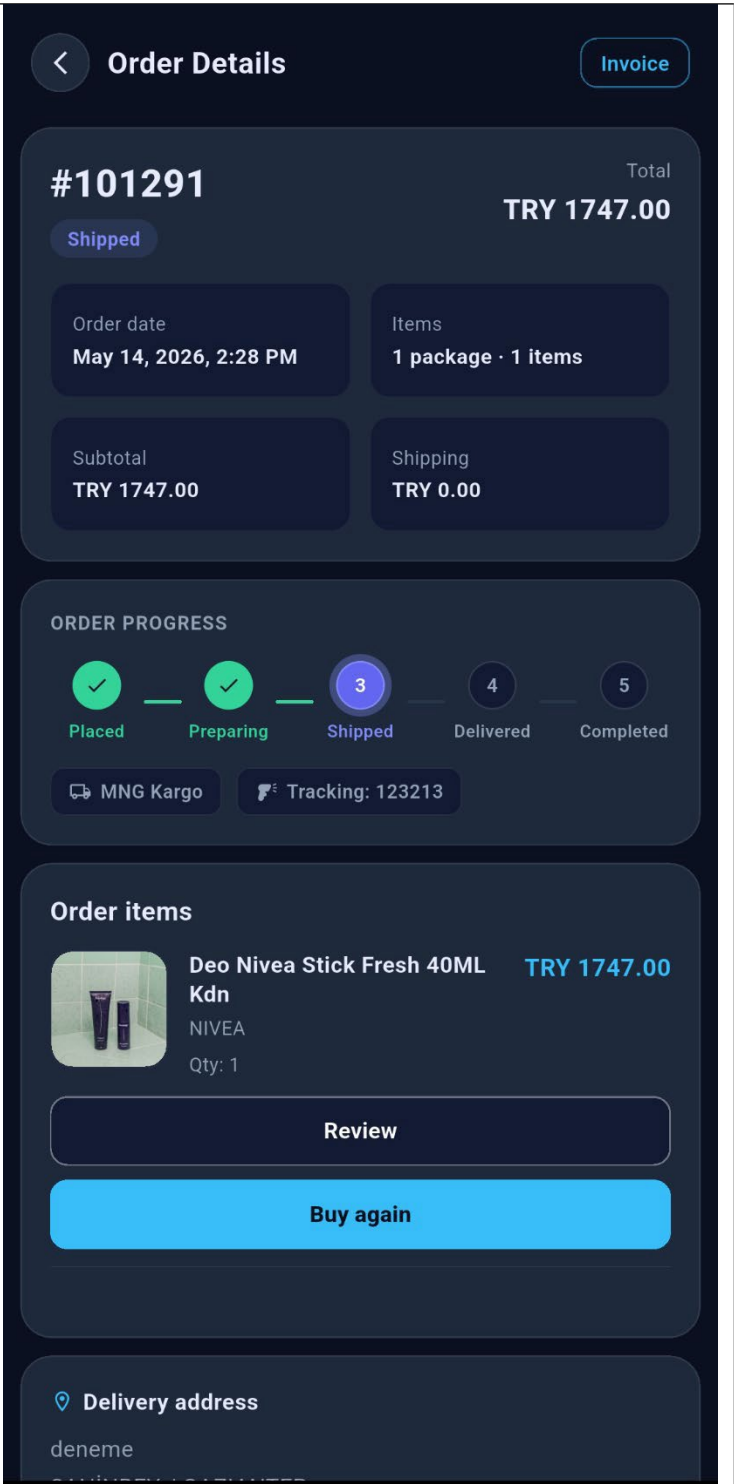


Figure 39 Mobile Order Details Page

Appendix Q – Additional Screenshots



Figure 42 Reviews Screen

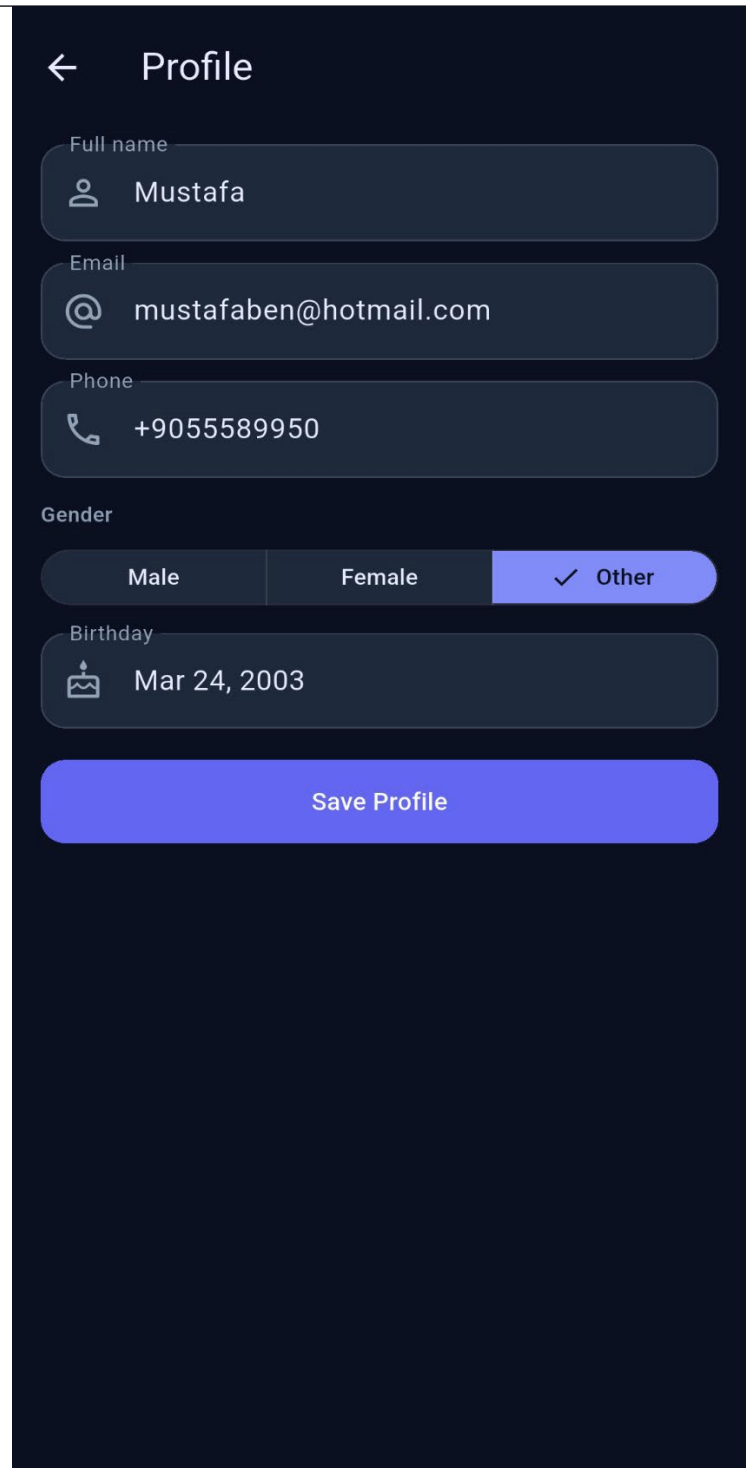


Figure 41 Profile screen

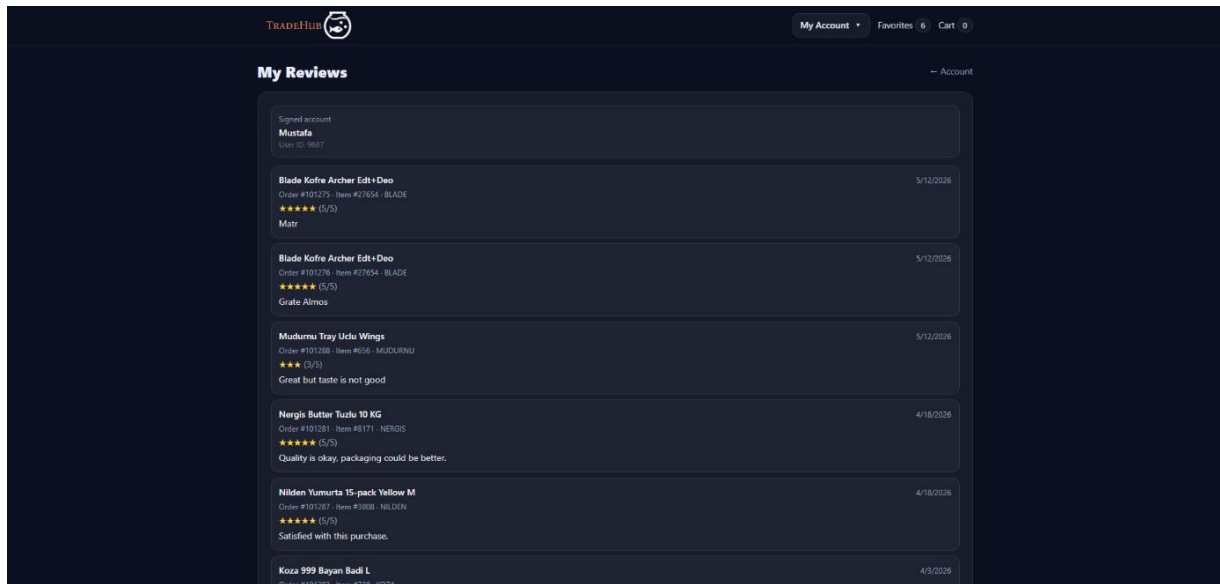


Figure 44 Review Page Web

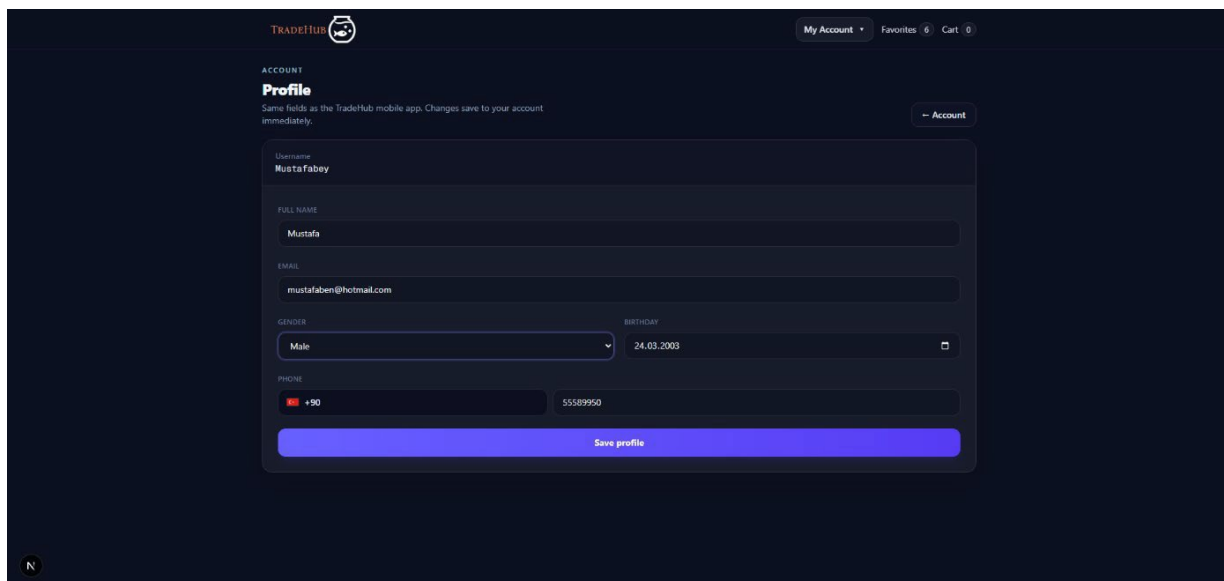


Figure 43 Profile Page Web

Appendix R – Proof of Work

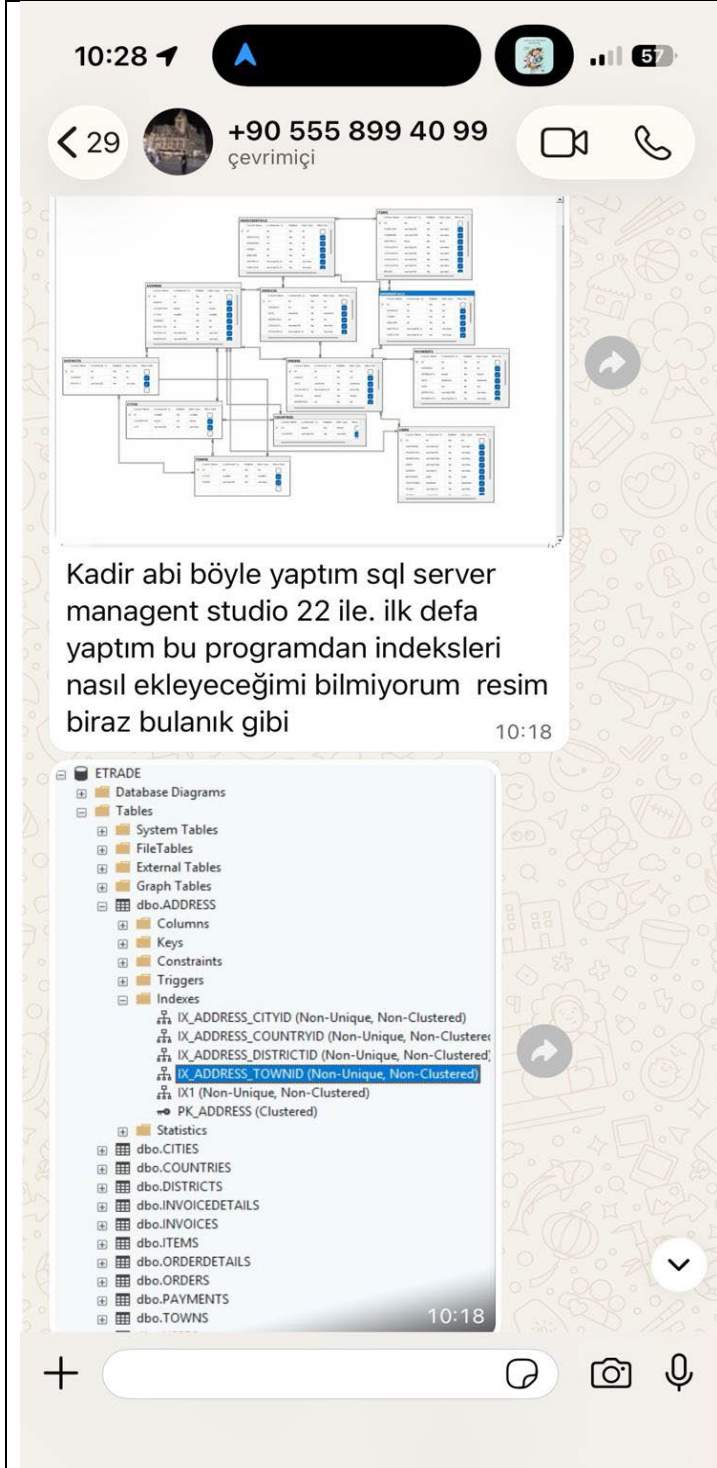


Figure 45 Database Feedback

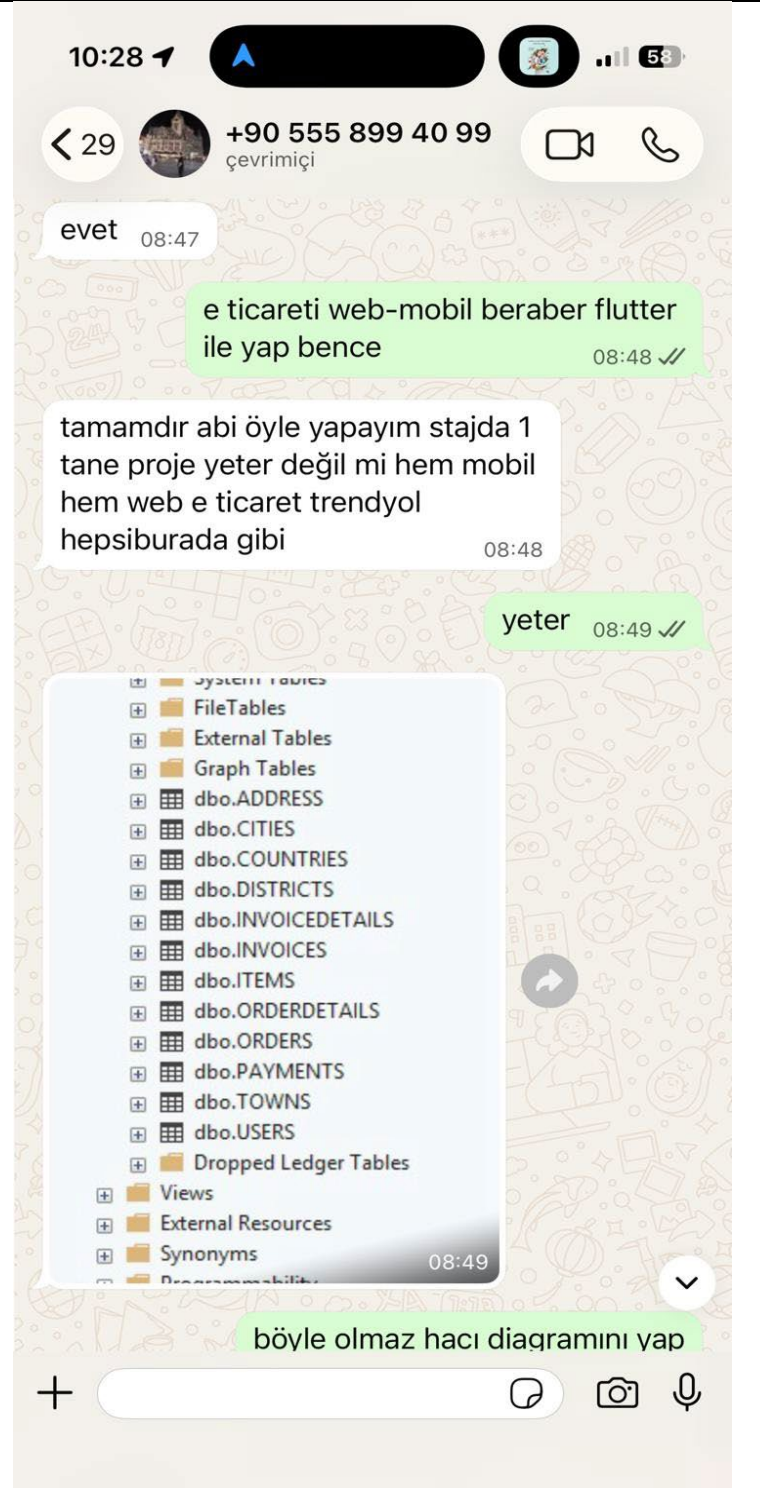


Figure 46 Develop both web and mobile platforms

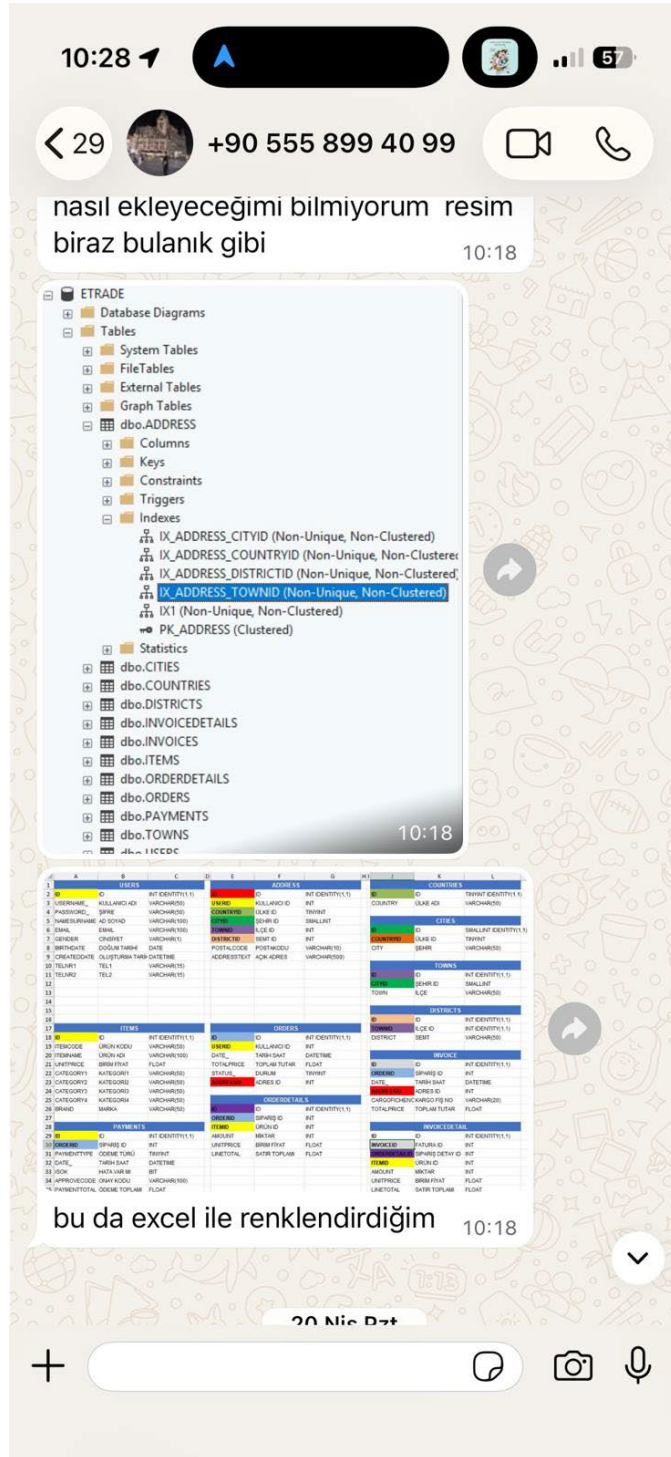


Figure 47 Database Schema



Figure 48 Intern working on the TradeHub project during the internship period



Figure 49 General view of the Adım Yazılım office workspace during the internship



Figure 50 Intern actively developing the TradeHub project at the company workspace



Figure 51 Gaziantep Teknopark building where Adım Yazılım is located



Figure 52 Entrance of Adım Yazılım office at Gaziantep Teknopark

Appendix S – Certificates



Figure 53 BTK Akademi Uygulamalarla SQL Öğreniyorum Certificate



Figure 54 Her Yönüyle SQL Server Certificate



Figure 55 BTK Akademi – React JS Next.js Certificate



Figure 56 Dart Programlama Dili Certificate



Figure 57 Flutter ile Mobil Uygulama Geliştirme Certificate